所在组别	"未来杯"2024 年第四届高校大数据挑战赛	参赛编号
本科组		bdc240366

基于多模态数据与深度学习的新能源电动汽车故障预警系统研究

摘要

我国新能源汽车产业在政策驱动下实现高速发展，产销规模已居全球首位，但质量管控滞后引发系列召回事件，凸显产业链发展失衡。本研究围绕故障预警体系开展创新探索：数据层开发多模态数据治理框架，破解充电静默期数据缺失，数据类别不对称性等难题；机理层构建"工况-故障-报警"因果推理模型，量化用户行为影响；算法层创新 Dual-Tower PredFormer 等架构，集成时空特征与物理约束降低误报率；应用层形成"充电优化-驾驶矫正-预防维护"多维管理策略，为提升车辆可靠性提供系统解决方案。

针对问题一：首先对数据进行缺失值填补与异常值修正等预处理操作，然后统计 1 至 9 月各月故障报警频次并分析其变化规律；结合充电状态、车速等指标，利用可视化手段揭示车主的充电习惯与用车行为特征。

针对问题二：采用互信息法、随机森林特征重要性评估等四种大数据方法，识别不同级别故障报警（0-3 级）的主要影响因素，明确每种报警等级最可能的故障类型，并探求了不同故障类型的关键特征。

针对问题三：问题三基于问题二结果，本文构建了增强型双塔架构深度学习模型和基于分层集成架构的深度学习模型两个模型，并通过交叉验证与 Val Acc 评估模型性能，最终选定 Dual-Tower PredFormer 最优的集成模型。

针对问题四：问题四应用问题三所构建该模型预测第 10 个月缺失的故障报警时刻及等级，输出具体时间点与报警级别。

针对问题五：问题五结合前述分析，在 Aprior 的关联分析的基础上 d 从充电策略、故障检修周期及用车模式优化等方面提出针对性建议，以降低故障风险并延长车辆使用寿命。

关键词：故障识别 随机森林 特征工程 多方法特征筛选 双塔模型

一、 问题重述

1.1 问题背景

我国新能源汽车产业的跨越式发展，植根于国家层面的前瞻性战略布局。早在1992 年，钱学森院士在《关于我国汽车工业发展新思路的建议》中明确指出：“要跳过燃油车阶段，直接进入新能源车时代，这是中国汽车工业实现弯道超车的唯一机会”。这一战略构想获得中央高度重视，2001 年《新能源汽车科技发展“十五”专项规划》首次将电动汽车列为国家重点研发方向，在购置补贴、充电基础设施专项债等政策协同推动下，我国新能源汽车产销量连续 9 年稳居全球首位。2024 年市场渗透率突破45%，充分印证了顶层设计的科学性与前瞻性（中国汽车工业协会，2024）。然而，市场规模的激增与技术迭代的加速也加剧了系统性风险。2024 年新能源汽车召回量创历史新高，达 126 万辆，其中电池热失控和电驱系统故障占比超过 65%。这种“发展速度-质量管控”的失衡，暴露了产业生态的深层矛盾（Wang et al., 2024）。

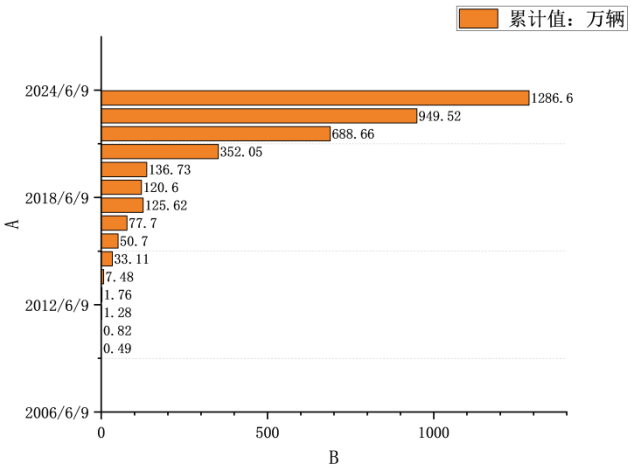

图 1　近年来中国新能源汽车销量图

燃油车与电动汽车的技术范式存在本质差异：前者依赖机械传动系统的精密协同，后者则依托“三电系统（电池、电机、电控）”的高度电子化集成。以热管理系统为例，燃油车余热回收效率通常不足 30%，而电动汽车需将电池组温差精确控制在±2℃以内，对 BMS（电池管理系统）的实时监测精度提出更高要求。技术范式的颠覆导致故障模式发生根本性转变，传统基于 OBD（车载诊断）的故障体系已难以应对高压绝缘失效、SOC（荷电状态）估算偏差等新型风险。但消费者对电动汽车的认知仍沿袭燃油车时代的经验框架。以小米 SU7 Ultra 用户调研为例，62%的受访者将“百公里加速时间”视为核心选购指标，而仅 19%关注电池健康度（SOH）监测功能。这种认知偏差引发使用行为失范：35%的用户习惯过度依赖快充（SOC>90%）或在低温环境急加速，加速电池极化效应。更值得关注的是，2024 年研究显示，78%的消费者认为“电动汽车自燃概率高于燃油车”，尽管权威数据表明其起火概率仅为燃油车的 1/3。这种安全偏见显著抑制潜在消费转化。从经济学视角看，新能源汽车市场呈现“创造性破坏”特征：厂商为抢占

份额，将产品迭代周期从 36 个月压缩至 18 个月，导致二手车残值率较同级燃油车低 40%。恶性竞争催生技术冒进，凸显技术成熟度与市场扩张速度的脱节。

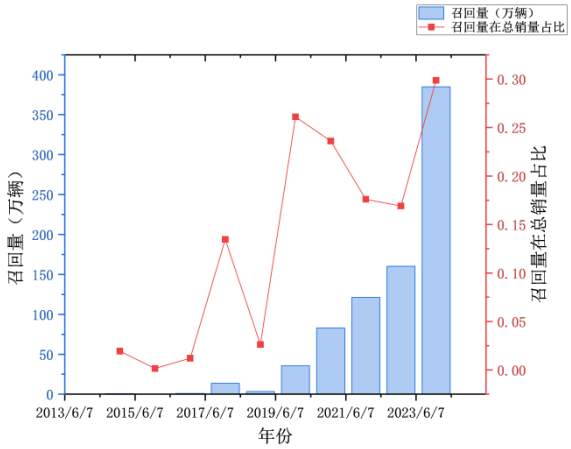

图 2　**近年来新能源汽**车召回量与其在当年新能源汽车总销量的占比

新能源汽车的普及已不可逆转，但规模化扩张必须与质量提升同步推进。建立故障预警模型不仅是技术问题，更是关乎行业生存与用户信任的战略选择。

1.2 问题提出

要求根据附件中某新能源车过去十个月的行驶数据进行总结和分析，然后探究造成该汽车不同级别报警的主要因素，并设计开发模型用于对该汽车未来的故障预警进行预测，最终提出使用建议。

问题一：基于附件提供的车辆多源时序数据先进行数据预处理，然后统计该汽车每个月发生的故障报警次数及其变化规律，并分析总结车主的充电用车规律。

问题二：针对附件中 0-3 级故障报警标签，根据附件数据选择合适方法找出造成电动汽车不同级别故障报警的主要影响因素。

问题三：，结合问题二的分析结果，构建多个电动汽车故障预警模型，并对模型进行比较和检验。

问题四：请根据问题三中建立的模型预测第 10 个月该车辆可能会出现的故障报警时刻。

问题五：综合前述分析结论，从多维度提出车辆运维策略。

二、 问题分析

2.1 问题一的分析

基于附件提供的车辆多源时序数据（包含车辆状态、电机参数及电池性能等共 20 项指标），需首先完成数据清洗与特征工程：首先对存在格式错误的数据采取删除冗余前缀的清洗策略，采用基于领域知识的多维度修正策略检测异常值并采用构建参数有效性规则库的方法修正异常值，结合回归方法填补缺失数据；统计 2022 年 1 月至 9 月各月

故障报警频次，分析其时间分布规律（如月度趋势、周期波动），并通过交叉谱密度图揭示故障与用车充电的关联性。进一步，通过折线图、热力图、雷达图等等可视化方法，从充电时段分布、SOC 变化曲线、单次行驶里程等维度解析展示车主充电、用车行为模式，展示了“充电-用车”协同特征。

2.2 问题二的分析

针对附件中 0-3 级故障报警标签，需构建多尺度影响因素挖掘框架：首先对数据进行预处理提高数据的完整性与一致性，然后构建特征工程，最后通过互信息法法、ANOVA 方差分析、随机森林特征重要性、聚类驱动特征筛选、物理模型特征验证五种方法各自独立筛选高关联特征组，最后后对各方法的筛选结果进行综合分析交叉验证得出结论。

2.3 问题三的分析

问题三要求结合问题二的分析结果，构建多个电动汽车故障预警模型并对模型进行比较和检验。首先在阅读前沿文献后确定选择建立增强型双塔架构深度学习模型和基于分层集成架构的深度学习模型两个模型。然后对两个模型进行检验。

2.4 问题四的分析

问题四要求根据问题三中建立的模型预测第 10 个月该车辆可能会出现的故障报警时刻。

2.5 问题五的分析

问题五要求结合前面问题的分析给出该车辆的使用建议，本文构建了一种基于 Aprior 的关联分析，最终从车主用车、汽车本身和定期故障检修三个角度给出使用建议。

三、 模型假设

为了便于问题的研究，对题目中某些条件进行简化及合理的进行假设：

1. 假设提供的行驶数据准确，不存在未记录的隐性故障因素或传感器偏差。
2. 假设训练集（1-9 月数据）与测试集（第 10 月数据）分布一致，未发生系统性变化（如电池老化突变）。
3. 假设模型输出的故障等级与时间点具有实际可解释性，可用于指导维护建议。
4. 假设历史数据中的故障模式对未来预测具有代表性。

四、 对问题一的求解

本文对第一问的求解思路如下：

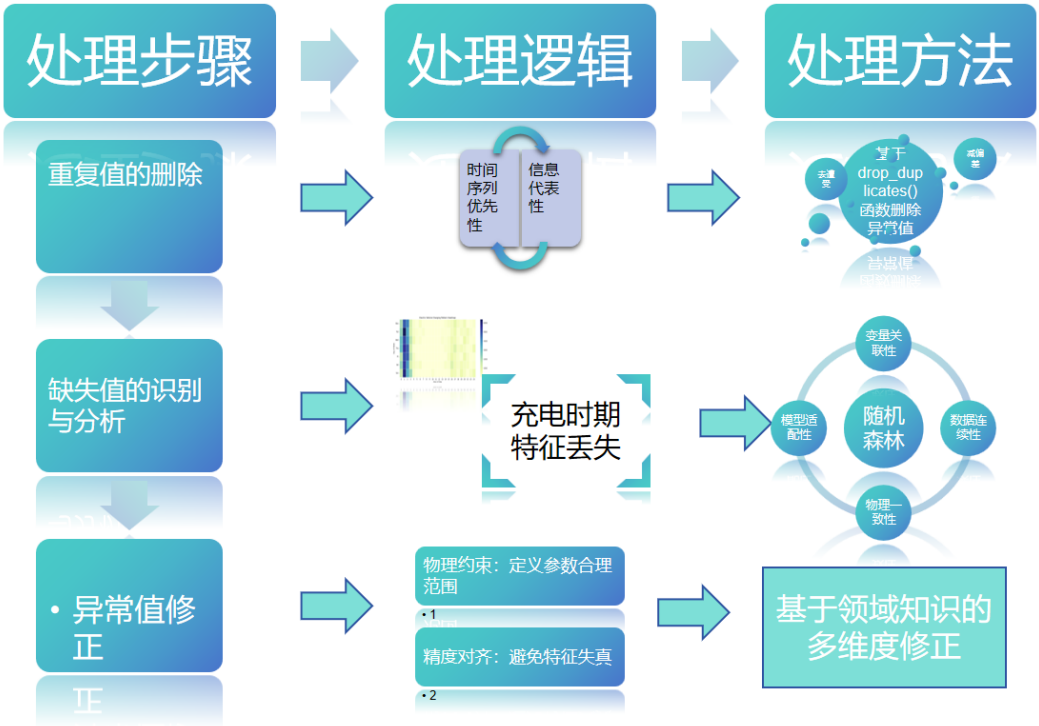

图 3 对第一问的求解思路

4.1 数据预处理

1.数据清洗

在原始数据中，部分字段因传感器采集或传输过程中的格式错误导致数据存储异常，存在冗余前缀（如“1：xxxxx”）。根据赛题数据说明，此类格式错误属于无效数据采集，需进行规范化处理。若保留错误格式，将直接影响后续数值型特征的统计分析与模型训练的准确性。针对上述问题，本文采用删除冗余前缀的清洗策略：对含冗余前缀（如“1：”）的字段，删除前缀并保留后续数值。例如，“1：1500”将修正为 1500，消除非必要字符干扰。通过以上处理，可确保数据的格式统一与数值有效性，为后续建模提供高质量输入，同时避免因格式错误或异常值引入的模型偏差。

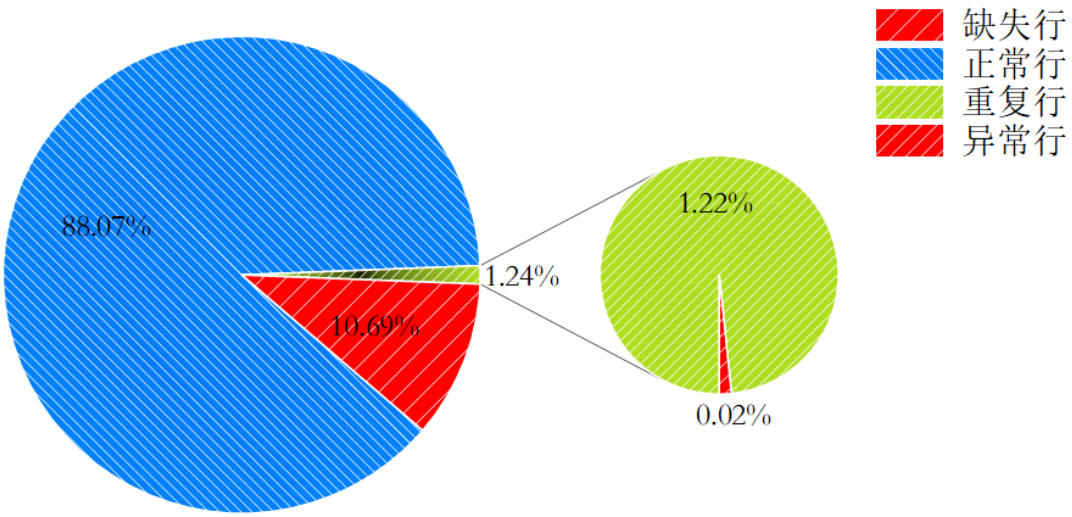

图 4 数据清理饼图

2.重复值的删除

重复数据的存在可能源于多种原因，如传感器故障、传输过程中的错误或记录错误等。这些重复数据的存在不仅会增加数据的冗余性，还可能导致数据分布的偏差，从而影响数据分析和模型构建的准确性。本文采用保留每组重复数据中的第一条记录，并删除其余的重复记录的异常值处理办法。这种方法的核心在重复的样本中采用是有效且具有代表性的第一条记录，而消除其他对模型训练或分析提供的信息量较少的重复数据的影响，减少噪声或误差的引入。通过这种方式，能够有效减少冗余数据，避免模型受到重复数据的影响。经实证，发现附件数据含有 14047 个重复行，修正后的数据集为后续模型的建立与优化打下了坚实基础。

3.缺失值的识别与分析

数据缺失可能由传感器故障、数据传输中断或极端环境干扰引起，破坏数据完整性并降低故障诊断模型的输入质量。经分析，附件数据中缺失值主要集中在充电时段，可能因部分传感器在充电过程中进入休眠模式所致，故本文采用回归方法填补充电期间的缺失数据。在模型构建中，首先基于电驱动系统能量传递与热力学特性对变量进行甄选，选择总电压、电池单体电压最高值与最低值作为核心预测变量，其依据在于：除这三项外，其他变量（如绝缘电阻、总电流）在缺失时无明显变化；总电压直接反映电池组输出能力，而单体电压极值则反映内部均衡状态，与电机参数变化密切相关；同时，剔除 SOC、绝缘电阻等冗余变量可避免共线性和采样不一致对预测精度的干扰。随后，利用随机森林算法构建多源特征协同回归模型，通过集成多棵决策树捕捉电池电压与电机参数间的非线性耦合（如电压-温度滞后效应），其 Bootstrap 采样和随机特征选择有效抑制局部噪声，并通过 Gini 指数量化验证了最高电压在电机温度预测中的主导作用，从而揭示了电池热失控与电机热管理的内在联系。实验证明，该方法成功修复了 740,574 条缺失数据，显著增强了数据时序连续性，使电机转速、转矩等参数与电池电压动态响应保持高度一致，为故障诊断模型提供了高质量、可解释且具统计鲁棒性的输入特征。

4.异常值修正

数据中的异常值可能由传感器测量偏差、极端工况下的瞬时波动或通信协议解析错误等引起。此类异常值会破坏数据的物理合理性和统计规律性，导致特征提取结果偏离真实车辆运行状态，从而影响故障诊断模型的泛化能力。在针对异常值的处理方面，本文采用基于领域知识的多维度修正策略。首先，依据车辆各子系统的工作原理和行业标准，定义关键参数的有效取值范围（如车速限制为 0-2200 km/h、SOC 限定为 0-100%），将超出物理可行范围的数据直接替换为缺失值；其次，考虑传感器的最小计量单元（如车速精度为 0.1 km/h、总电压精度为 0.1 V），对数值进行标准化调整，通过四舍五入消除因数据采集精度引起的微小偏移。这种双重修正机制既能剔除明显违背物理规律的数据，又能统一数值精度，避免因量化误差导致的特征失真。

具体实现中，本文通过构建参数有效性规则库，逐字段检测异常值并动态修正。对于离散型参数（如车辆状态码），直接过滤非定义枚举值；对于连续型参数，采用边界截断与最小步长对齐相结合的方式，确保修正后的数值既符合物理约束，又满足工程应用的精度要求。

经实证，发现附件数据中有 237 行是含有异常值，修正后的数据有效恢复了工况参数的逻辑一致性，电流波动、温度梯度等关键物理量的动态耦合关系符合机电系统固有特性。为后续构建预测模型提供了兼具数据可靠性与算法泛化性的基准数据集。

4.2 规律总结

4.2.1 充电规律

（一）充电频次角度的分析

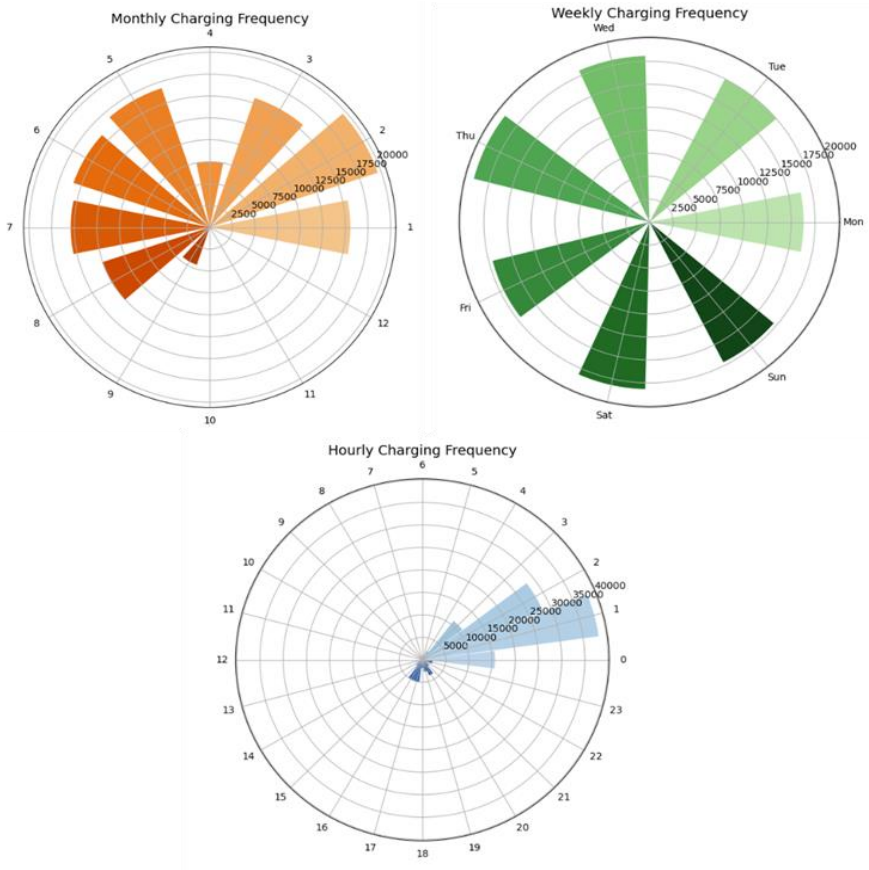

图 5 充电频次图

图 5 利用极坐标图展示月度、周度、小时维度的充电频率，通过扇形大小和颜色突出充电频次分布，从而定位不同时间的充电特征。

在小时维度（蓝色极坐标图），0－1 点和 2－3 点是明显的充电高峰时段，这与夜间电价低谷相吻合，也契合“用车结束后集中补能”的日常习惯；深夜车辆通常处于闲置状态，充电既不影响白天出行，又能节约费用。

在周际维度（绿色极坐标图），周五、周六、周日的充电频次显著高于工作日，这与是由于工作日通勤需求较高的现实状况相符合，工作日中车辆工作时间多，车辆在工作结束后用于充电的时间相对较短，而周末时间相对空闲，适于集中补电。

在月度维度（橙色极坐标图），3－8 月的充电频率整体较高，尤其在 6－7 月达到峰值，与此阶段车辆使用频率上升相吻合，或与暑期出行及空调使用增多等因素有关，进一步说明用车量的增加会直接带来充电需求的上升。

综合而言，这些充电分布特征既反映了车主对经济性（夜间低电价）、实际用车需求（工作日通勤）及季节性变化（夏季出行高峰）的综合考量，也为后续分析和优化充电策略提供了重要参考。

（二）充电时段角度的分析

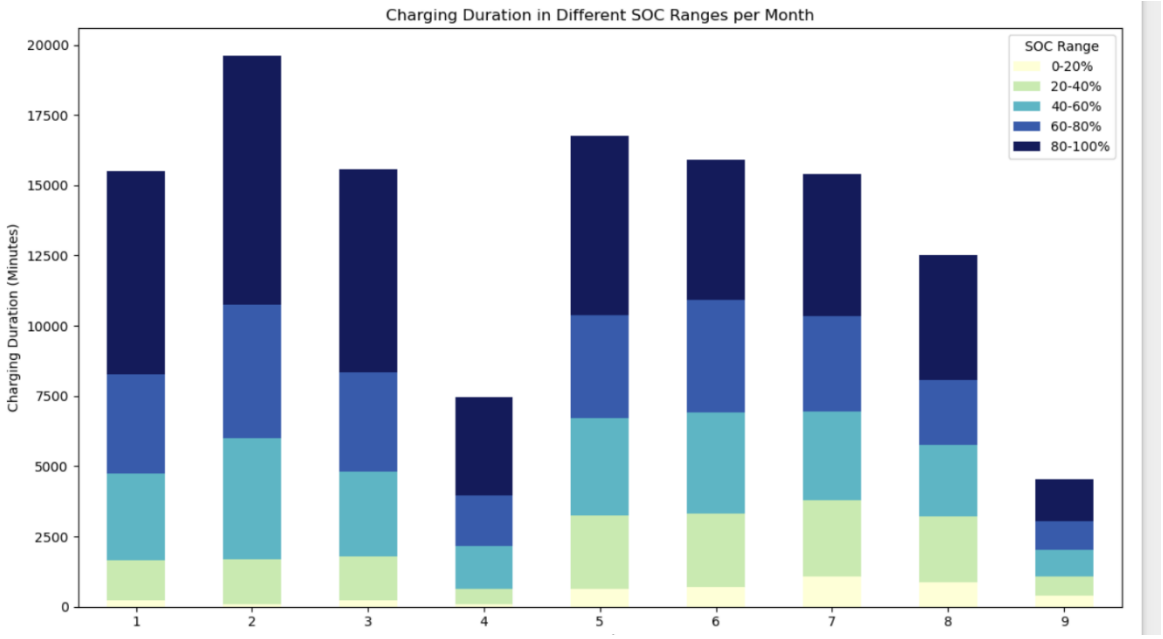

图 6 SOC 月份分布图

图 6 展示了不同 SOC（State of Charge，电池充电状态）区间在每个月的充电时长分布，采用堆叠柱状图形式，直观呈现各 SOC 范围的充电时长占比。纵轴表示充电时长（分钟），横轴代表月份，颜色区分不同 SOC 区间，其中最深蓝色（80－100%）占据主导，说明车主在高 SOC 范围内充电的时间较长。每个月的充电时长存在波动，2 月和 5 月的充电时长最高，而 4 月和 9 月相对较低，可能反映出季节性变化或车主用车频率的差异。此外，SOC 区间的分布较为稳定，40－80%区间的充电时长占比较大，而 0－20%区间的充电时长较少，表明车主倾向于在电量较高时补充电能，而非深度放电后再充电。

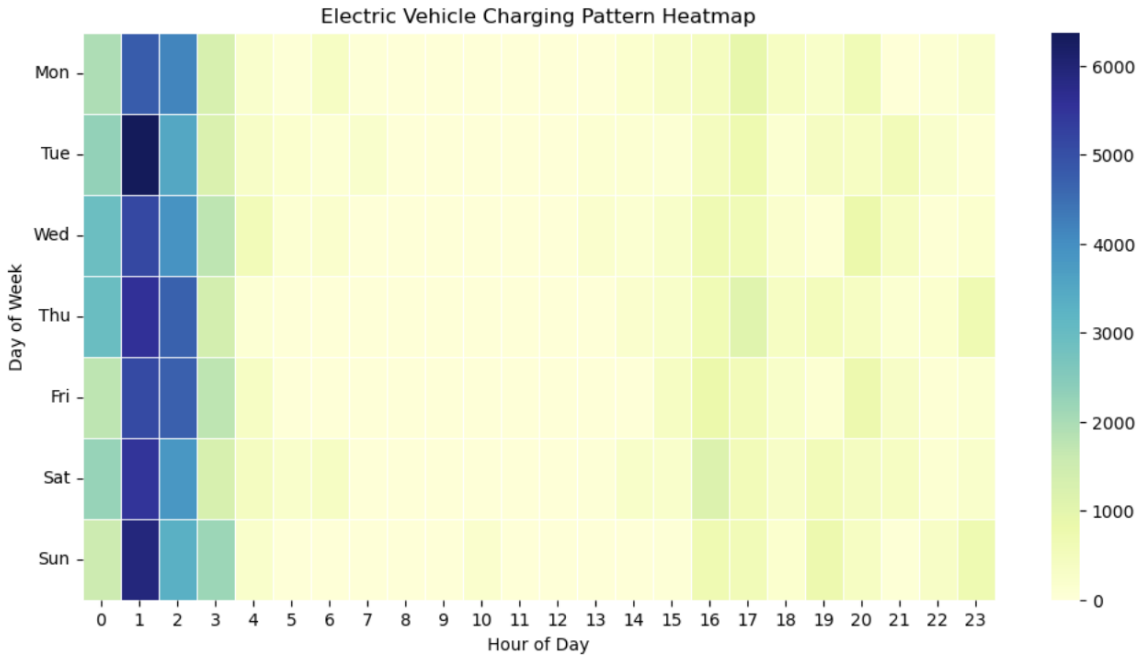

图 7 充电频率图

图 7 为**电动汽车充电模式热力图**，展示了车主在一周七天的不同小时内的充电频次

分布情况。横轴表示一天中的 24 小时，纵轴表示一周七天，颜色深度代表充电频次，深蓝色表示充电最频繁的时段，浅黄色则表示充电较少或无充电行为。图中显示，车主的充电行为高度集中在凌晨 0－3 点，特别是 1－2 点达到峰值，周二凌晨 1 点的充电频次最高。白天和晚间的充电频率明显较低，整体呈现**夜间充电为主、白天充电较少**的模式。此外，周一至周五的凌晨充电频次较稳定，表明车主在工作日采用较为规律的充电习惯，而周末的充电频次略有减少，可能与不同出行需求或休息日安排有关。

综合分析，该车主倾向于在凌晨 1－2 点进行充电，可能的原因包括以下两点：**1. 利用非活跃用车时段**，凌晨是低出行需求时段，车辆大部分时间处于闲置状态，在此时充电可避免占用白天用车时间，提高使用效率；**2. 降低充电成本并优化能源利用**，电力部门通常采用分时电价策略，凌晨电价较低，车主选择该时段充电可有效减少成本，长期来看经济性更强。

4.2.2 用车规律

（一）用车频次角度的分析

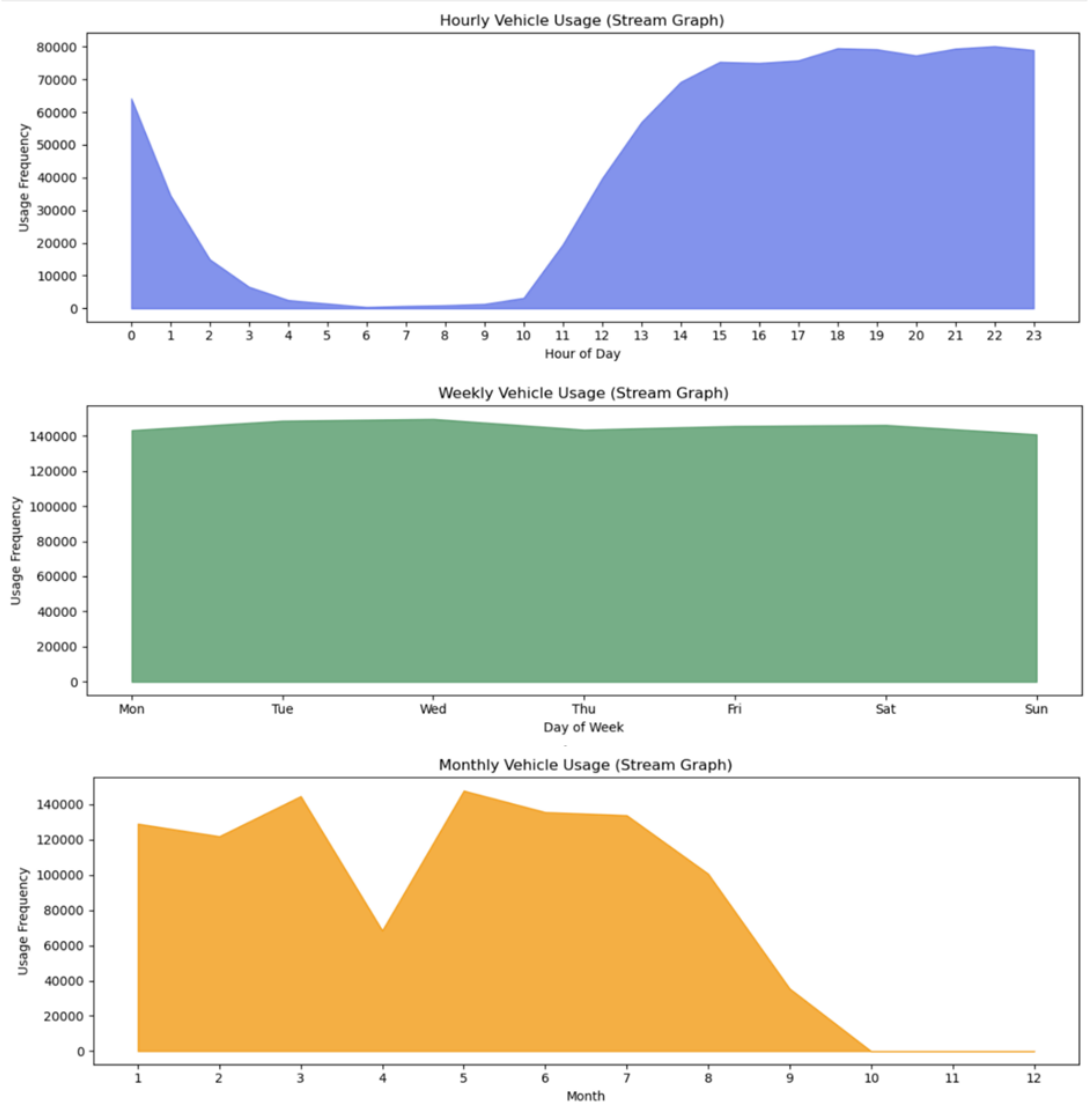

图 8 用车频次图

图 8 通过面积图展示了小时、周、月三个维度的用车频次变化，使高低峰时段的对比更加直观，并有效呈现周期性规律。

从小时维度来看，每日用车呈现明显的早晚高峰，**0－1 点用车频次较高**，对应夜间返程需求；**12 点后用车量逐步上升，17－23 点达到峰值**，符合通勤和晚间出行模式；而凌晨 **4－10 点用车频次最低**，处于日常用车的低谷期。

从周维度来看，每周七天的用车频次整体保持稳定，**无明显的工作日与周末差异**，表明车主的出行需求较均衡，未表现出特定的周末或工作日出行偏好。

从月份维度来看，**1－3 月和 5－7 月的用车频次较高**，而 **4 月的用车频次明显较低**，这一趋势可能与季度初、年中出行需求的集中有关，例如假期、季节性天气变化或年度规划导致的行程调整。

综合来看，该图清晰地揭示了车主的用车模式，并为进一步优化出行策略或能源管理提供了重要参考。

（二）用车时段角度的分析

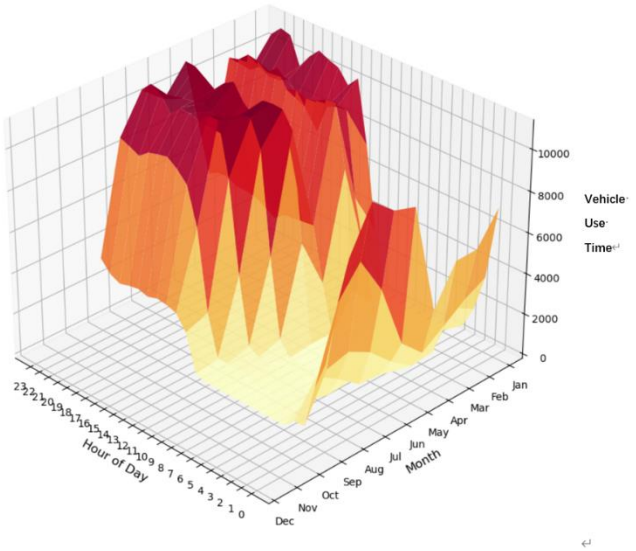

图 9 用车时间山岳图

图 9 通过三维曲面图将月份、小时、用车时长融合，以用车时间山岳图生动展示多变量关系，直观呈现不同时间的用车时长堆积与波动。

从小时维度分析，早晚高峰特征明显，**早上 8 点和晚上 18 点的用车时长显著堆积**，形成多个突出的“山峰”，反映出车主在通勤时段的高频用车行为。这一特征在多个月份均持续存在，说明早晚高峰是全年稳定的用车模式。与此同时，凌晨时段（如 0－5 点）和中午（12－14 点）的用车时长相对较短，表现为较低的“山谷”区域，表明这段时间车辆处于低活跃状态。

从月份维度来看，不同月份的用车时长存在显著变化，**部分月份整体用车时长较高，形成较大的“山脉”**，而某些月份则整体较低。例如，5－7 月的用车时长高峰更为陡峭，表明该阶段的出行时间较长，与月度用车频次图的趋势一致，而 4 月的整体用车时间相对较低，进一步验证了该月的用车活跃度下降。这种月度波动可能与季节、气候变化、假期安排或车主个人出行计划有关。

综合来看，**图 9 不仅揭示了车主用车时长的时间分布特征，也展现了不同月份和时段之间的动态变化**。早晚高峰的“山峰”结构稳定存在，反映出通勤需求的规律性，而

月度的波动则表明用车强度会随时间推移而变化。该图形象地描绘了用车时长的积累模式，为进一步优化用车规划、充电策略和能源管理提供了直观的数据支持。

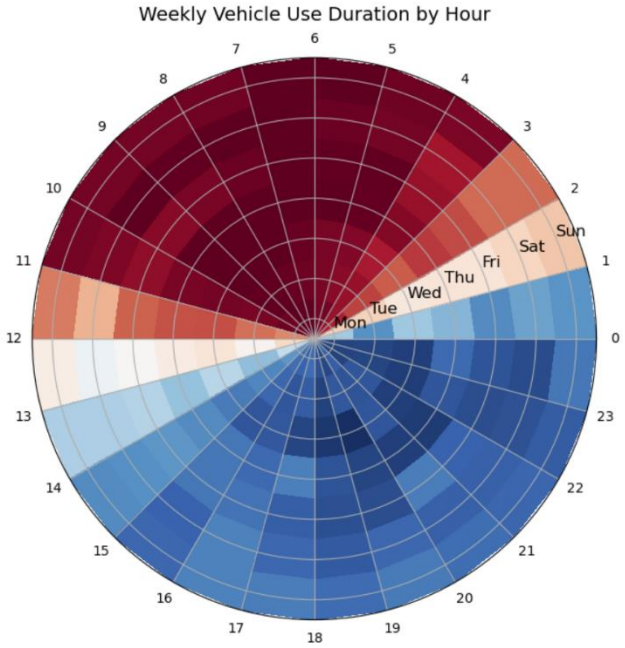

图 10 **每周每小**时用车极坐标热力图

图 10 通过极坐标热力图结合周、小时维度，通过颜色深浅传递用车时长强度，环形布局整合时间周期，热力对比强化视觉认知，快速定位高频用车时段。

从**时间角度**来看，不同时间段的用车需求存在显著差异。通常，**早高峰**（6-11 点）和**晚高峰**（18-23 点）是用车时长较长的时段，这些时段与上班和下班的通勤出行密切相关，交通负担较重，因此需要通过深红色等较深的颜色来标示这些高峰时段。相比之下，**午间时段**（13-17 点）的用车需求较低，通常是工作或休息状态，使用蓝色或浅蓝色显示较为合适。另一个显著的差异是**周末**与**工作日**的用车模式不同。**周末**（特别是周六和周日）某些时段，如**早 8 点和晚 20 点**，用车时长更为突出，通常与休闲和社交活动相关，反映了人们在周末的出行需求更集中，可能是购物、休闲娱乐或外出探亲的原因。相比之下，工作日的用车需求则较为平稳。

从**月度角度**来看，季节和天气的变化会导致月度用车需求波动。通常**冬季**（如 12 月到 2 月）和**夏季**（如 6 月到 8 月）的用车时长较长，寒冷或炎热的天气增加了人们对交通工具的依赖。**春秋季节**的用车需求则较低，这与气候宜人的特点及节假日分布有关。此外，节假日如**春节**、**国庆**等假期常常会导致某些月份用车时长激增，特别是在**春节前后**，人们的返乡和旅行需求会带来用车高峰。

总体来看，通过结合**时间**和**月度**两个维度的分析，全面解释用车趋势，识别高峰时段与季节性变化。

（三）行驶里程角度的分析

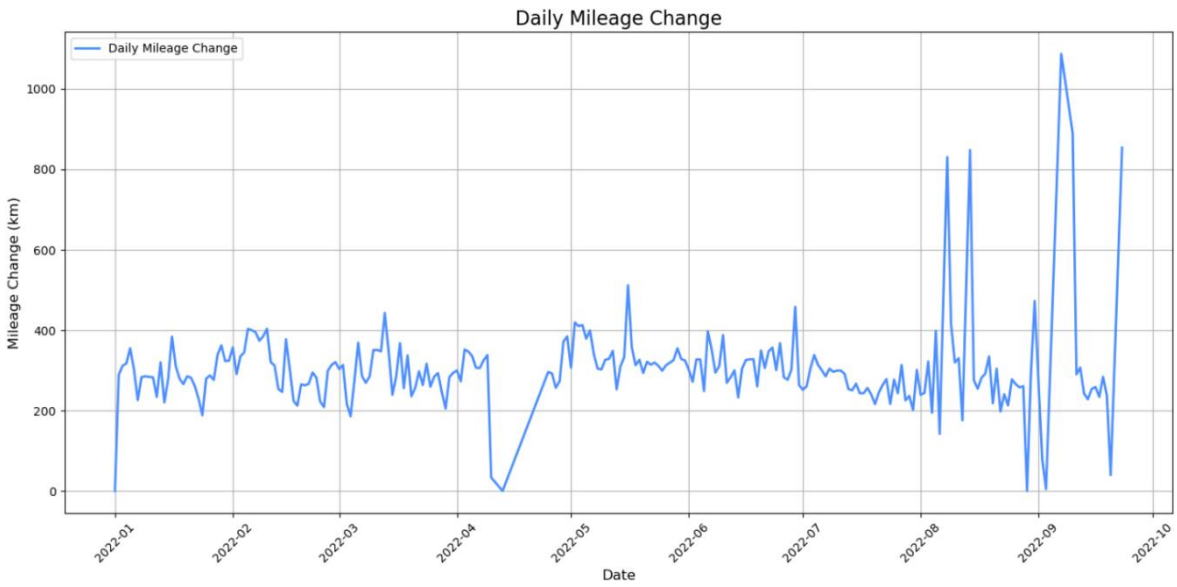

图 11 每日行驶距离

图 11 通过折线图以时间为轴，简洁展示每日里程变化趋势，清晰捕捉数据波动细节，便于分析里程变化的阶段性与偶发性特征。

每日行驶里程的波动表现出明显的**阶段性特征**，具体体现在 2022 年 1 月到 7 月以及 8 月到 10 月两个阶段。首先，从 **2022 年 1 月到 7 月**，车主的行驶里程相对平稳，波动范围通常在 200 至 400 公里之间。这一阶段的平稳波动可能反映了日常通勤或常规出行的模式，车主的出行行为较为稳定，且大部分时间的里程数较为均衡，通常是日常生活中的短途或中等距离行驶，例如上下班通勤、购物或偶尔的家庭出行。

然而，从 **8 月到 10 月**，行驶里程突然出现多次**骤增**，特别是在一些日子里，行驶里程超出 800 公里，甚至突破 1000 公里，这表明在这一阶段，车主的出行需求发生了显著变化。可能的原因包括长途出行需求和偶发性和阶段性集中特征等

综合来看，这一现象反映了车主出行需求的**阶段性波动**，即在某些特定的时期（如暑期、假期或长周末），车主的出行需求会急剧增加，导致行驶里程的大幅度波动。而在其他时段，日常的短途出行则会使得行驶里程保持相对平稳。这种波动模式强调了车主的出行行为并非固定不变，而是受到不同社会、季节、假期等因素的影响，表现出强烈的**周期性**和**阶段性**特征。

4.2.3 故障规律

（一）电池故障频次分析

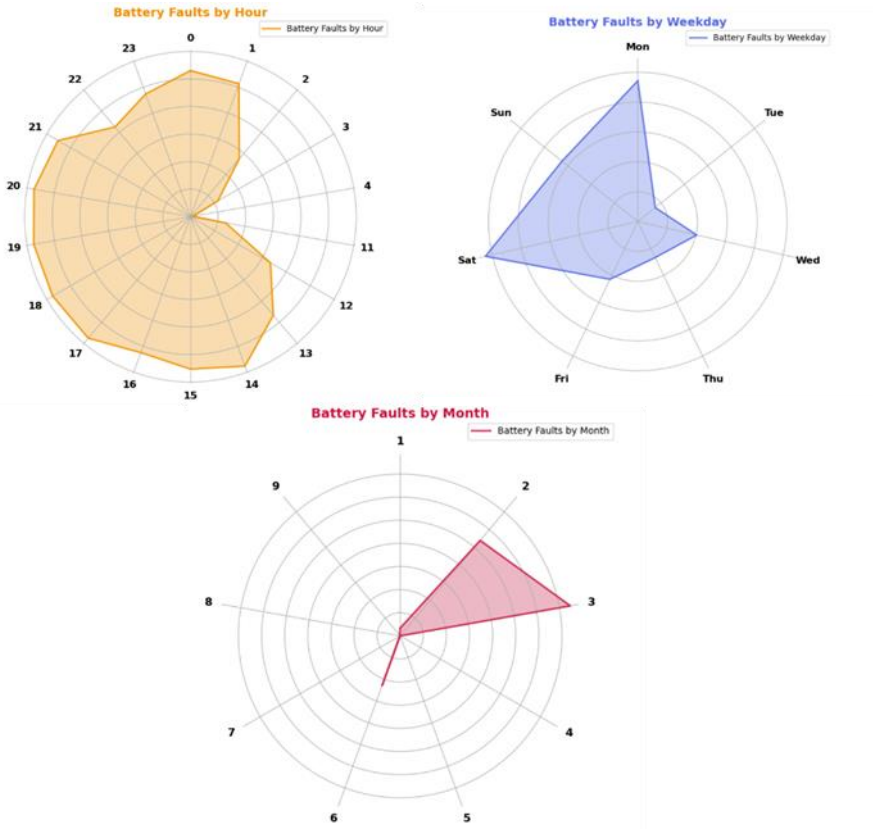

图 12 电池故障雷达图

图 12 通过雷达图分别展示电池故障在小时、星期、月份三个时间维度的分布情况，每个维度以极坐标形式呈现数据大小。可直观对比同一主体（电池故障）在不同分类维度下的分布差异，快速定位高故障时段。

从**小时维度**来看，故障高发时段主要集中在 **23 点至次日 1 点**，可能与**夜间高负荷使用**或**长时间充电后**的使用状态相关。此外，**16 点至 22 点**也表现出较高的故障概率，可能是由于傍晚和夜晚的**通勤高峰期间**，电池负荷加重。

从**星期维度**来看，**周一、周六和周日**故障明显高于其他工作日，尤其是**周一**，可能与**周末后的出行需求增加**有关，而**周三和周四**则表现出较少的故障发生，可能是因为这两天的使用频率较低。

从**月份维度**来看，**1 至 3 月**为故障的**高发期**，可能与**寒冷天气**导致电池性能下降有关，而其他月份（4 月到 12 月）的故障概率显著降低，**温暖天气下电池性能更为稳定**。通过这些维度的分析，可以有效识别电池故障的规律性并采取针对性的**预防措施**。

（二）电池故障时间分析

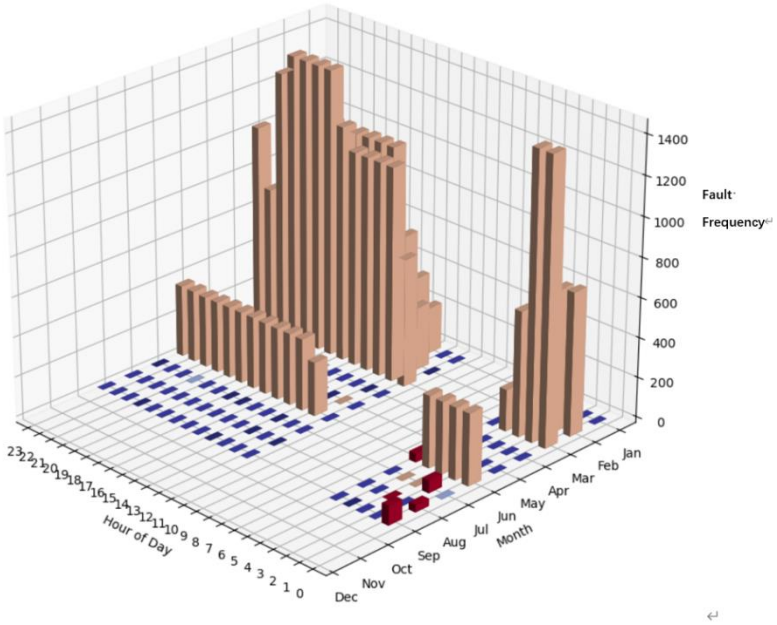

图 13 故障频率三维柱状图

图 13 通过三位柱状图分别展示电池故障在小时、月份两个时间维度下的故障分布情况，故障等级通过颜色区分，越接近红色说明故障等级越高，蓝色则代表没有故障。

从**小时维度**分析来看，发现故障的发生在**夜间时段**尤为突出，尤其是 **23 点至次日 1 点**这一时段，故障柱体的高度明显较高。夜间气温较低，电池处于高负荷或低温状态，尤其是长时间的充电过程中，电池可能因温度过低而出现故障。此外，夜间的高负荷使用也可能导致电池在充电过程中的电流波动加剧，从而引发故障。

从**月度分析**来看，**2 月和 3 月**的白天时段（尤其是早晨和午后）也出现了较高的故障频率。这反映了**冬季寒冷天气**对电池的持久影响，尽管白天气温有所回升，但寒冷的天气和早晚温差仍对电池的性能产生了不容忽视的负面影响。电池在经历过一整夜的低温后，在早晨或午后的高负荷使用下，容易发生故障。

总体而言，时间维度中的**季节性因素**、**日夜温差**以及**小时和月份的交叉效应**共同作用，导致了冬季特别是在夜间及特定月份（如 2 月、3 月）出现较高的电池故障频率。这一规律的识别为后续的电池管理和**预防维护策略**提供了重要参考，能够针对季节变化及时间段特征，制定更加精确的电池保养和故障预警机制。

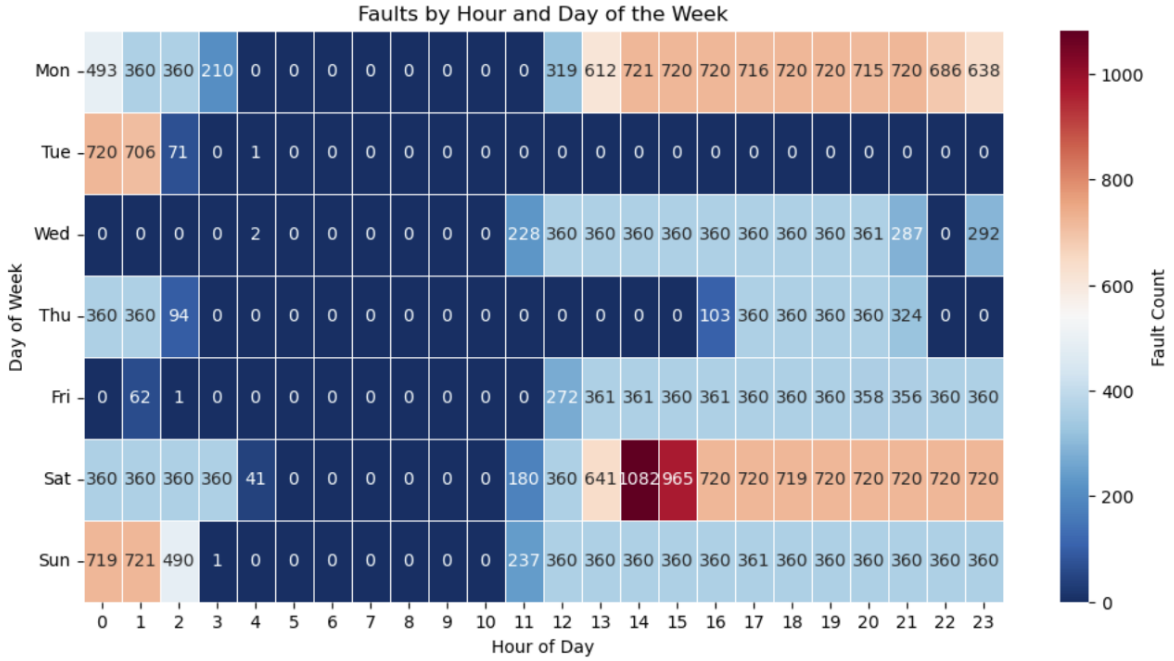

图 14 基于星期与小时的故障频次热力分析图

图 14 以“星期”为行、“小时”为列，通过颜色深浅（热力值）展示故障频次分布。能够直观呈现二维数据的密度分布，高故障区域通过颜色深度一目了然。

分析发现，**周六 14-15 点**的故障频率最高，达到了 **965 次**，这可能与周末用车高峰有关，尤其是在长时间使用电池后，电池故障的概率增大。**周日**和**周二**的凌晨（如 0 点、1 点）也有较高故障频率，可能是夜间长时间用车或周末后电池恢复时发生的故障。相比之下，**周三**和**周四**的故障频率较低，这些天通常是工作日的中间，出行需求相对较少，电池负荷也较轻，因此故障发生较少。

（四）基于交叉谱密度图的分析

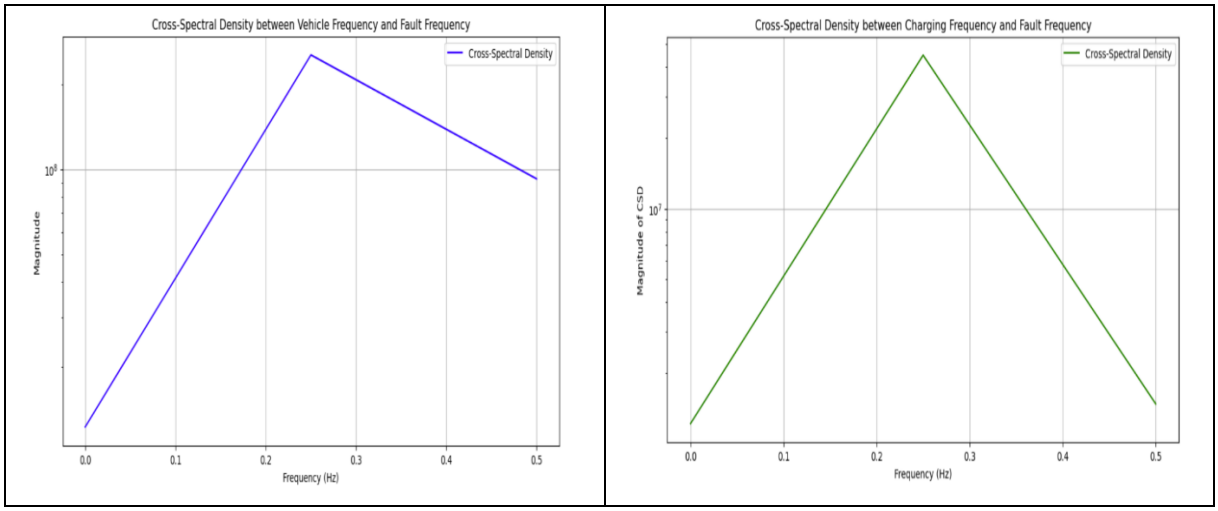

图 5 用车充电和故障的交叉谱密度图

交叉谱密度（Cross-Spectral Density，CSD）是分析两个信号在频域中相关性的工具，常用于检测信号之间的关系。csd 函数用于计算用车频次和故障频次之间的交叉

谱密度。它的输出包括频率（f）和交叉谱密度（Pxy），其中 Pxy 表示在特定频率上的信号关系强度。

图表显示了频率与交叉谱密度之间的关系，其中 f 表示频率（单位：赫兹，Hz），并展示了交叉谱密度的幅度（大小）。该图表揭示了用车频次与故障频次之间的频域相关性。

通过研究用车频率与故障频率的交叉谱密度图，发现约 0.25Hz 的频率点处交叉谱密度值达到峰值。这表明在这个特定频率下，车辆使用频率与故障频率的相关性最强，即在这个频率区间内，故障发生的概率较高。

同样，研究充电频率与故障频率的交叉谱密度图时，也在 0.25Hz 处观察到谱密度的峰值。这表明充电频率与故障频率在这个特定频率点上存在显著相关性，暗示充电行为可能在某些频率范围内诱发故障。

五、 对问题二的求解

问题二要求找出造成不同级别故障报警的主要影响因素。针对此问题本文设计了一种多维特征归因分析方案，通过该方案能从海量运行参数中识别不同报警等级的核心驱动因素，为故障预警与性能优化提供可靠依据。整个方案涵盖数据预处理、特征构建、多维度分析与可视化呈现四大环节，形成了一个从原始数据到主要因素判别的完整链路。

5.1 数据预处理：

附件中提供的原始数据中包含时间戳、车辆状态、电池参数、电机性能等二十余项基础指标。首先按时间顺序整理数据，确保时序逻辑连贯。然后针对数据缺失问题，采用分类型与数值型差异化的填补策略：对于车辆状态、充电状态等类别型参数，使用前后向填充保证状态连续性；对于车速、电压、温度等数值型指标，结合移动窗口均值与前后填充法，在保留局部趋势的同时消除缺失干扰。最后对异常值处理则采用动态阈值策略，对多数数值特征基于改进分位数法（5%与 95%）结合标准差法设定合理范围，超出阈值的异常值以中位数替代，避免极端值对后续分析的干扰。通过预处理，数据完整性与一致性显著提升，为后续分析奠定基础。

5.2 特征工程

在基础参数的基础上，通过物理关系推导、时间序列分析与统计计算构建了四十余项衍生特征，能够全方位刻画车辆运行状态。例如，通过电池单体电压极值计算电压差异与一致性指数，反映电池组均衡状态；通过 SOC 变化率与能耗率，反应能量消耗效率；结合电机转速、转矩构建负载率与热负荷指标，评估电机工作强度；利用时间差序列提取急加速、急刹车频率等驾驶行为特征。此外，还引入了滚动窗口统计量捕捉参数波动规律，构建温度梯度、温升加速度等动态指标，以反应热管理性能。这些特征从电池系统、驱动系统、热管理系统、驾驶行为四个维度构建了完整的车辆状态表征体系。

部分特征工程如下：

$\Delta T = \frac{total_seconds(diff(D))}{3600}$	《基于 TDOA 数据采集和传输效率的优化方法》 发布来源：信息记录材料

	作者：杨青、刘军
$\Delta V = \max(V_i) - \min(V_i)$	《串联锂离子电池组电压巡检电路》 发布来源：电子设计工程 作者：全立地、徐静云、王玉龙、蔡志端
$I_{charge} = \begin{cases} I, 当 S_{charge} = 1 \\ 0, 当 S_{charge} \neq 1 \end{cases}$	《蓄电池充电电流控制的LLC谐振变换器研究》 发布来源：《电测与仪表》 作者：邓吉利、曹太强、陈汝兵、杨雨雪
$R_{T-n} = \dfrac{T}{n+\varepsilon}$	《永磁同步电机直接转矩控制转矩调节器设计》 发布来源：中国电机工程学报 作者：杨建飞、胡育文
$T_{low-SOC} = cumsum(\Delta t \times 1(SOC < 20))$	《基于机器学习的纯电动汽车SOC估算方法研究》 作者：甘志恒
$CV_{col-name} = \dfrac{std(rolling(col, size))}{mean(rolling(col, size)) + \varepsilon}$	《基于变异系数的交通流序列突变检测方法》 作者：蒋大治、张亮亮

5.3 **多方法特征**筛选

针对预设的四个报警等级，本文采用四种分析方法交叉验证探究对报警等级影响的关键特征：

5.3.1 **互信息法**

该方法是一种基于信息熵理论衡量特征与目标变量间的非线性关联度，通过计算特征与各报警等级之间的互信息得分，识别对特定等级故障具有显著信息增益的特征，适用于捕捉复杂工况下的非线性耦合关系的方法。

互信息法的数学原理如下：

$$I(X;Y) = \sum_{y \in Y} \sum_{x \in X} p(w,y) \log\left(\frac{p(x,y)}{p(x)p(y)}\right)$$

式中，I（X,Y）为互信息值，p(w,y)表示特征 X 与目标 Y 的联合概率分布， p(x) 和 p(y) 为边缘分布。该公式的本质是通过对数似然比捕捉变量间的协同变化模式——当 X 与 Y 独立时， p(x,y)=p(x)p(y)，此时互信息值为零；而当两者存在强关联时，联合分布与边缘分布的乘积差异增大，导致互信息值显著上升。因此 I(X;Y) 值越大，表明特征 X 对目标 Y 的信息增益越显著。

在具体操作层面，本文先计算各特征与目标变量之间的互信息值，按得分降序排序后，筛选出信息量最大的前 5 个关键特征。

互信息法的优势在于无需线性假设，可捕捉非线性关系，且对数据分布无严格要求，能够兼容代码中混合了离散状态（如充电状态）和连续数值（如电压、温度）的特征类型，从而适配复杂车辆数据的分析需求。

5.3.2 ANOVA 方差分析（F-test）

ANOVA 方差分析（F-test）是一种基于组间差异比较的统计方法，用于评估特征在不同报警等级间的线性可分性。其核心原理是通过计算特征在各报警等级组间的方差与组内方差的比值（F 值），量化特征对目标变量的区分能力。其数学表达式为：

$$\mathrm{F}=\frac{\text{组间方差}}{\text{组内方差}}=\frac{MS_{between}}{MS_{within}}$$

式中，$MS_{between}$ 和 MS_{within} 分别为组间均方和组内方差，组间均方反映不同报警等级间的特征差异，组内均方表征同一报警等级内的数据离散度。F 值越大，表明特征区分不同故障等级的能力越强。

在具体实现中，本文计算各变量 F 统计量及对应 p 值，保留 F 值排名前 5 位且通过显著性水平 α =0.05 检验的特征。

该方法的优势在于计算效率高，能够快速筛选出与目标变量存在显著线性关系的特征（如车速、总电流、温度差异等），并通过 P 值提供统计显著性检验结果，此特性使其适用于高维数据的初步特征筛选，为后续多方法融合分析提供基础支撑。

5.3.3 随机森林特征重要性（Random Forest）

随机森林特征重要性（Random Forest）是一种基于决策树集成学习的特征评估方法，其核心原理是通过基尼不纯度（Gini Impurity）的下降量量化特征对分类决策的贡献度。

在具体操作层面，本文计算各变量贡献度得分后，保留前 5 位高贡献特征。

随机森林的优势在于能够自动捕捉特征间的非线性关系（如电压-电流比的阈值效应）和交互效应（如总电流与最高温度值的协同影响），且对高维度、多共线性数据具有鲁棒性。

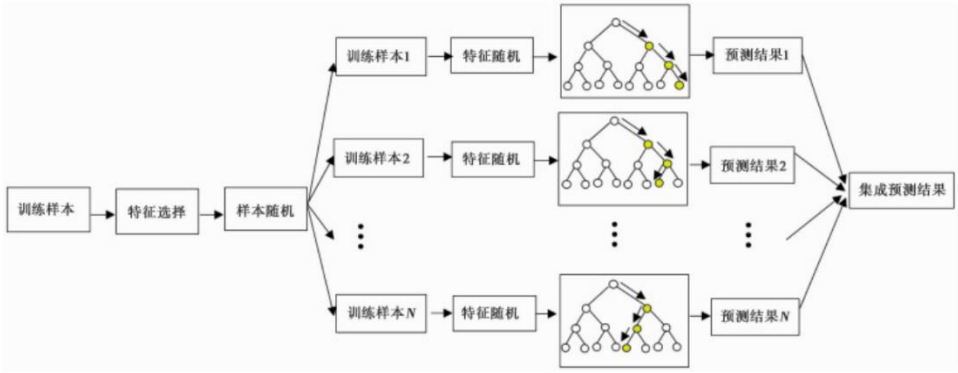

图 16 随机森林算法原理图

5.3.4 聚类驱动特征筛选（K-means Clustering）

聚类驱动特征筛选（K-means Clustering）是一种基于无监督学习的特征评估方法，其核心思想是通过数据内在分布模式识别关键影响因素。该方法首先对标准化后的特征矩阵进行聚类分析（代码中使用 K-means 算法），通过轮廓系数（Silhouette Coefficient）衡量聚类质量以确定最优簇数。

在具体操作层面，本文通过遍历簇数范围（2 至 5），选择轮廓系数最大值对应的簇数作为最优划分，随后计算每个特征在各簇中的均值绝对差异，差异越大表明该特征对簇间分离的贡献越显著（如总电压、SOC 变化率等）。最终筛选差异值排名前 5 的特征作为关键影响因素。

此方法的优势在于无需依赖目标标签（如报警等级），能够从数据分布中挖掘隐含关联（如电池压差变化率与温度梯度的协同模式），且通过轮廓系数避免人工设定簇数的主观偏差。

输入：输入数据 维度 Dim
输入数据个数 N。

过程：函数 K-Means(X,K)

```
1. 随机生成 K 个 Dim 维的中心点；
2. while 算法未收敛 do
3.     for N 个点 do
4.         计算每个点属于哪一类；
5.     end for
6.     for K 个中心点 do
7.         1. 找出所有属于自己这一类的所有数据点；
8.         2. 把自己的坐标修改为这些数据点的中心点坐标；
9.     end for
10. end while
```

输出：K 个聚类中心及样本划分结果.

5.3.5 物理模型特征验证

这是一种基于领域知识与物理机理构建特征的方法，其核心原理是通过预先定义与系统运行机制直接相关的工程参数（如电池压差、电机热负荷、温升加速度等），确保关键物理指标在数据驱动分析中不被遗漏。

物理模型特征验证的优势在于直接映射至工程参数，支持故障根因分析，且避免纯数据驱动模型忽略关键机理特征。

本文依据电动汽车动力系统的运行特性，构建了多个机理导向特征：例如通过电池单体电压最高值与最低值的差值定义电压差异，反映电池组内部一致性状态；基于总电压与总电流乘积计算功率输出，量化能量转换效率；利用驱动电机温度与转速的比值定义电机热负荷，表征电机热管理状态；结合 SOC 差分与时间间隔计算 SOC 变化率，描述电池荷电状态的动态特性。这些特征直接映射至电池、电机、热管理等子系统的核心物理量。

该模型的优势在于通过先验知识锁定与故障强相关的工程参数，避免纯数据驱动方法因统计偏差忽略低贡献度但机理显著的指标能够为故障根因分析提供可解释的工程依据，与互信息法、随机森林等数据驱动结果形成双重验证。

每种方法独立生成特征排名后，统计各特征在不同方法中的出现频次，综合筛选出高频次指标。通过这种多方法交叉验证机制可以有效降低单一分析方法的主观偏差，确保结论的稳健性。

5.4 处理结果

最终的求解结果为：报警等级为 1 的故障类型最有可能是动力电池故障，推测其主要影响因素为最高温度值、SOC 变化率、电压-电流比等；报警等级 2 的故障类型最有可能是控制系统故障与电机故障，推测其主要影响因素为最高温度值、SOC、总电压等；报警等级 3 的故障类型最有可能是充电系统故障与综合系统故障，推测其主要影响因素为电压-电流比、SOC、总电压等。

具体处理结果如下：

报警等级	主要影响因素	最可能故障类型
等级 1	最高温度值、SOC 变化率、电压-电流比、最大单体电压波动、SOC、总电压、总电流等	动力电池故障
等级 2	最高温度值、SOC、总电压、总电流、电机温度-转速斜率、电压一致性指数、转矩波动等	控制系统故障、电机故障
等级 3	电压-电流比、SOC、总电压、总电流、最高温度值、低 SOC 运行时长、功率输出、超速运行时长等	充电系统故障、综合系统故障

根据上文明确的对应各报警等级的最可能故障类型，对其进行关键特征的探究具体结果如下表：

故障类型	关联特征编号	关键特征示例
动力电池故障	3,4,5,6,7,8,9,10,52,53	电压差异、SOC 变化率、R0_EKF（内阻估计）
控制系统故障	17,25,26,27,28,30,31	转矩-转速比、效率、相关系数（温度与电机温度）
电机故障	27,28,29,32,33	温度-转速斜率、转矩波动、热负荷
充电系统故障	19,20,21,22,23,24	充电功率、充电效率、频繁充放电次数
空调系统故障	12,13,14,16,44	温度差异、温升加速度、高温运行时长（间接）

根据上表结果，在电动汽车的故障诊断中，报警等级用于指示故障的严重程度和类型。根据不同的报警等级，可能的故障类型和主要影响因素如下：

报警等级 1：动力电池故障

动力电池作为电动汽车的核心部件，其性能直接影响车辆的续航能力和安全性。在报警等级 1 中，最可能的故障类型是动力电池故障，主要影响因素包括最高温度值、SOC（电池荷电状态）变化率、电压-电流比、最大单体电压波动、总电压和总电流等。这些因素直接影响电池的性能和安全性。例如，电池温度过高可能导致热失控，SOC 变化率异常可能预示电池容量衰减。

报警等级 2：控制系统故障和电机故障

控制系统和电机是电动汽车的关键部件，其故障会影响车辆的动力输出和操控性能。在报警等级 2 中，最可能的故障类型是控制系统故障和电机故障，主要影响因素包括最高温度值、SOC、总电压、总电流、电机温度-转速斜率、电压一致性指数和转矩波动等。这些指标用于评估控制系统和电机的运行状态。像是电机温度与转速的斜率变化可能指示电机过载或冷却系统故障，转矩波动可能反映电机控制策略不稳定。

报警等级 3：充电系统故障和综合系统故障

充电系统和综合系统的故障可能影响车辆的充电效率和整体性能。在报警等级 3 中，最可能的故障类型是充电系统故障和综合系统故障，主要影响因素包括电压-电流比、SOC、总电压、总电流、最高温度值、低 SOC 运行时长、功率输出和超速运行时长等。这些因素主要影响充电系统和整体系统的可靠性。例如，电压-电流比异常可能指示充电系统故障，低 SOC 运行时长过长可能加速电池老化。

通过监测上述关键特征指标，可以及时发现潜在故障，采取预防措施，确保电动汽车的安全运行。

六、 对问题三的求解

6.1 模型介绍

在阅读研究了 Yujin Tang、赵景远的文献后，认为基于增强型双塔架构的深度学习模型与基于分层集成架构的深度学习模型的两种深度学习模型适应于附件数据的多模态混合、动态变化特征，因此选择构建这两种模型对问题三进行求解。

6.1.1 基于增强型双塔架构的 Dual-Tower PredFormer 深度学习模型

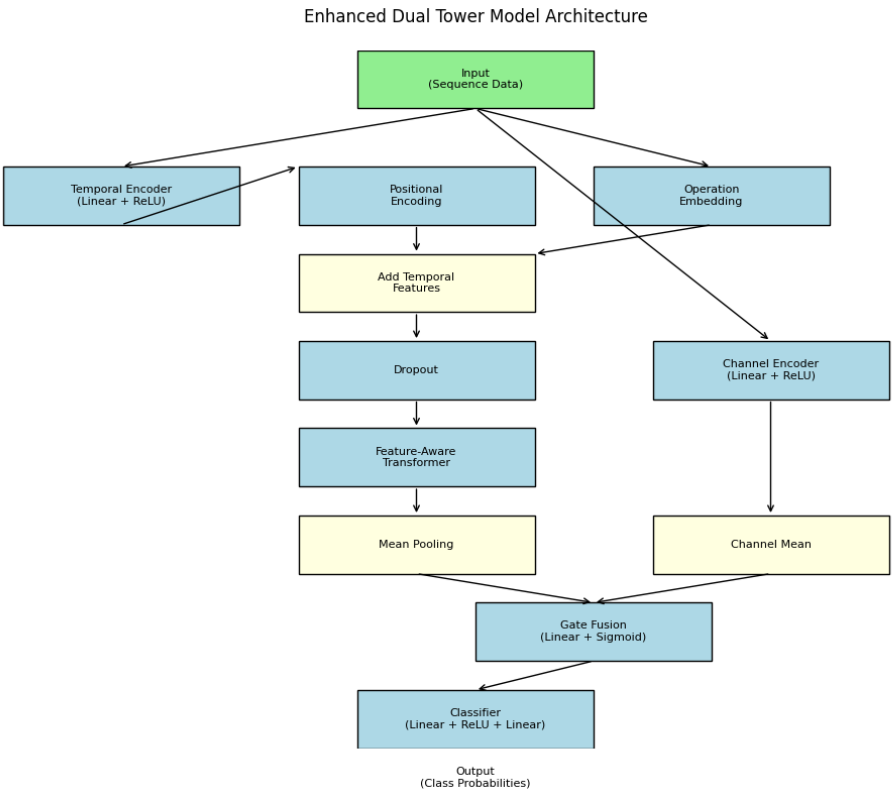

图 6 增强型双塔架构深度学习模型结构图

PredFormer 提出了一种基于纯 Transformer 架构的时空预测模型，通过全注意力机制突破传统卷积神经网络（CNN）固有的归纳偏置限制，展现出更高的可扩展性与泛化能力。其模型设计围绕时空分解 Transformer 编码层展开，核心由两个互补的注意力分支构成：时序动态编码分支采用时间优先因子化编码器（Fac-T-S），通过滑动窗口注意力机制精确捕捉电压波动、温度趋势等时序信号的动态演变规律；空间关联编码分支则依托空间优先因子化编码器（Fac-S-T），利用跨步长注意力挖掘多时间步间横向数据的协同变化模式，进一步揭示全局序列的潜在特征。

在此基础上，PredFormer 创新性地引入 Gated SwiGLU-Transformer 模块，将多头时空注意力机制与基于 SwiGLU 的门控前馈网络相融合。该模块通过动态调节信息流，在特定场景（如高温告警）中自适应增强温度相关特征的注意力权重，从而显著提升模型对突发异常状态的感知能力与响应敏感度。此外，PredFormer 通过交错层与因子化配置优化不同任务的空间-时间分辨率需求。最终，模型利用门控融合机制自适应整合时空分支的特征贡献，在保留时序细节与全局模式平衡的基础上，实现特征表达的优化与统一。

该架构的具体训练步骤如下：

双塔增强模型架构的训练过程步骤	
Step1	初始化模型参数，并行构建时序编码器和通道编码器，分别赋予随机权重。
Step2	输入时间序列数据，时序编码器提取时间维度的动态变化特征（如电压波动、温度趋势），通道编码器挖掘跨时间步的关联模式（如全局协同变化）。
Step3	根据实时报警等级，通过特征感知模块动态调整特征权重，增强关键信号（如高温报警时聚焦温度特征），削弱次要信息。
Step4	将两个编码器的输出输入门控融合机制，通过自适应权重分配（如 Sigmoid

	门控）平衡时序细节与全局模式的贡献，生成融合特征。
Step5	利用分类器对融合特征进行多级报警概率预测，反向传播更新模型参数，优先优化误分类样本的特征表达。
Step6	迭代训练至收敛，最终模型通过协同时序与通道特征、动态特征筛选及智能融合，实现复杂工况下的精准状态预警。

6.1.2 基于分层集成架构的深度学习模型

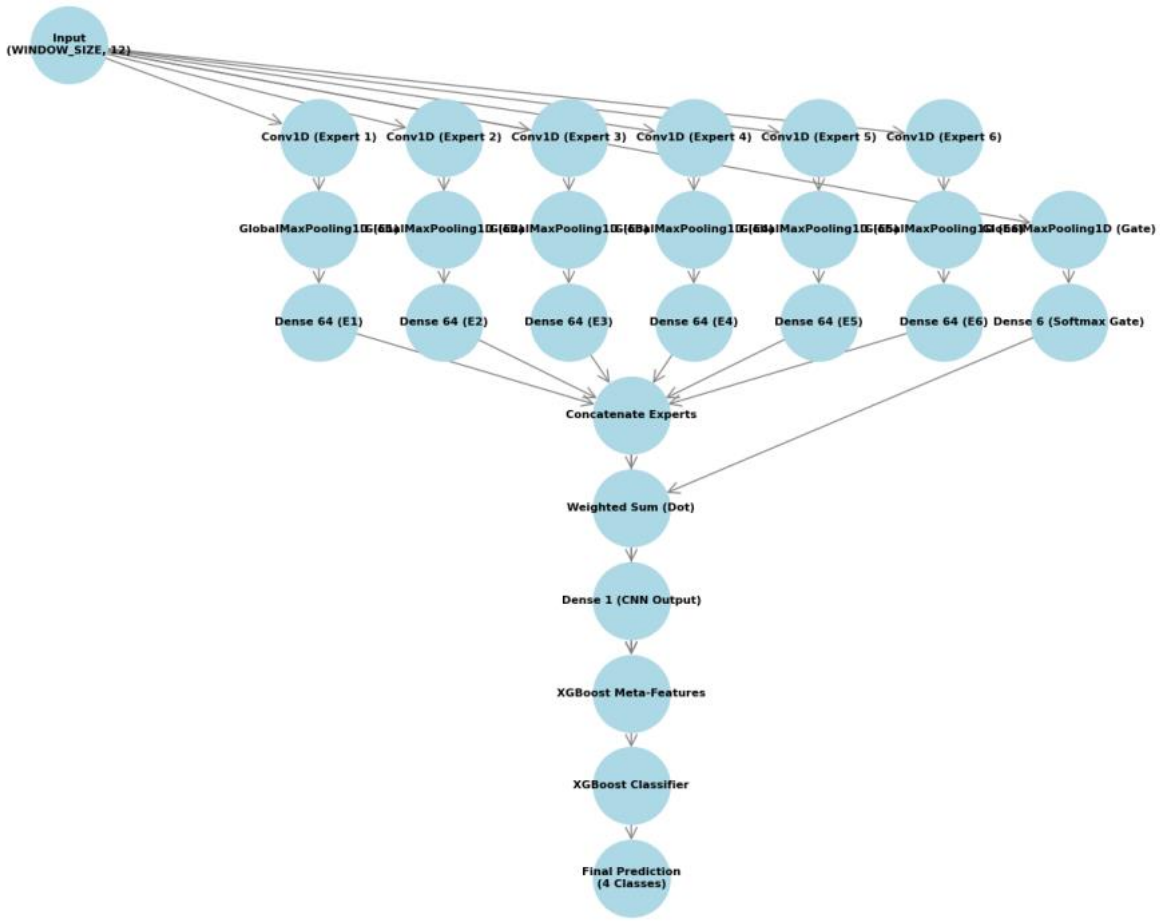

图 7 分层集成学习架构结构图

分层的预测性维护集成学习系统架构通过多阶段协同实现设备状态的智能监控与预警。首先从多源数据中采集原始信号，然后经过数据清洗，过采样，调整不同报警等级权重与异常修正模块剔除噪声并降低错误率，生成高质量的训练数据集；随后系统利用特征注意力机制动态识别关键参数（如温度骤升、电压异常波动等），结合多级报警规则筛选出对故障敏感的指标，并通过共享参数机制协调不同子模型的学习过程，确保特征表达的一致性。在集成决策层，系统根据预设阈值对设备工况进行分级评估，结合概率分析预测潜在故障风险，最终通过自动化决策模块输出维护建议。整个架构通过数据优化、特征强化与模型协同的闭环流程，实现从原始数据到精准预警的全链路智能化管理。

其组件交互图如下：

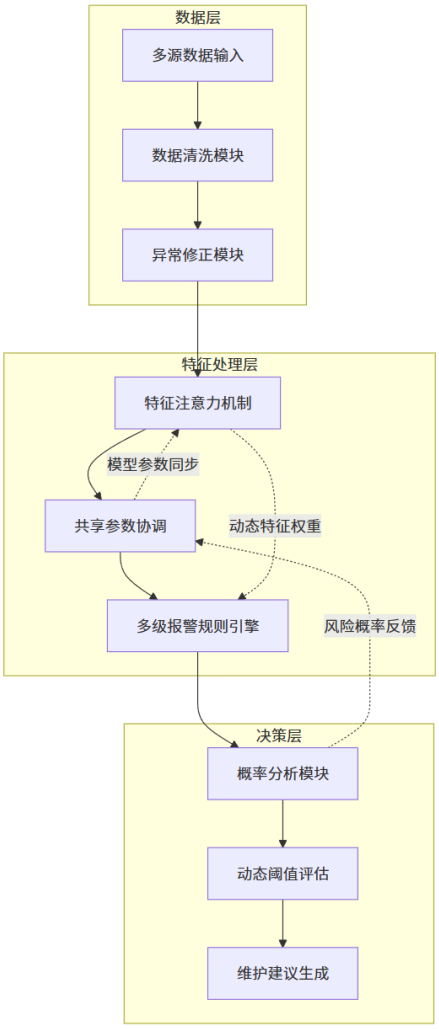

图 8 分层的集成学习架构的组件交互图

6.2 模型检验

本文采用 Val Acc 作为模型的检测指标。

在机器学习中，Val Acc 是评估模型在验证集上分类性能的核心指标，该指标能够反应模型在陌生数据上的能力，Val Acc 是判断模型繁华能力和优化策略的重要依据。其计算公式如下：

$$\text{val acc} = \frac{TP + TN}{TP + TN + FP + FN}$$

其中 TP（真阳性）、TN（真阴性）为正确分类的样本，FP（假阳性）、FN（假阴性）为误分类样本。

对本文构建的两个模型以及三个传统的 Val Acc 检验结果如下表：

模型	双塔模型	分层集成架构模型	CarBoost	LightGBM	XGboost

Val Acc	0.8956	0.8340	0.8521	0.7975	0.8638

对以上结果进行观察发现本文构建的双塔模型效果最好，故采用双塔模型对后续问题四进行求解。

七、 对问题四的求解

根据问题三中建立的双塔模型对数据进行预测的部分结果如下：

编号	数据采集时间	报警等级
1	2022/10/2 19:07	2
2	2022/10/2 19:08	2
…	…	…
3	2022/10/6 2:45	1
4	2022/10/6 2:46	1
5	2022/10/6 2:47	1
…	…	…
6	2022/10/6 2:56	3
7	2022/10/6 2:57	3
……	……	……

表 1 问题四预测结果

结合问题二的影响因素分析，预测结果包含 1-3 级报警，其中：等级 1（轻度故障）：如动力电池故障；等级 2（中度故障）：如控制系统故障、电机故障；等级 3（严重故障）：如充电系统故障、综合系统故障。预测时间覆盖 10 月的不同日期和时段。实证表明本文构架的 Dual-Tower PredFormer 模型能够有效捕捉故障信号，实现对故障的精确预测

八、 对问题五的求解

8.1 基于 Apriori 关联分析的故障发掘

Apriori 关联分析是一种用于挖掘数据中频繁项集与关联规则的经典算法，其核心思想是通过逐层搜索和剪枝策略，利用先验知识（频繁项集的子集必须也是频繁的）高效发现高频组合，并基于支持度与置信度生成关联规则。

该算法的流程图如下：

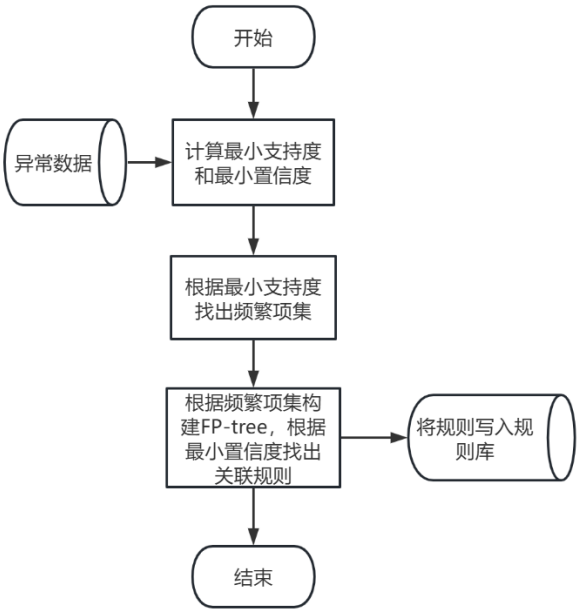

图 20 Aprior 算法流程图

通过该算法对数据集进行处理后本文得到的具体结果如下：

	样本数	支持度	置信度
Low_SOC (低 SOC)	1,156,029	0.049	0.216
High_Motor_Controller_Temperature（高电机控制器温度)	1,156,029	0.076	0.265
High_Motor_Temperature(高电机温度)	1,156,029	0	0
High_Motor_Speed(高电机转速)	1,156,029	0.098	0.297
Charging(充电状态)	1,156,029	0.107	0.309

对以上结果进行分析可知，导致故障出现的前五个特征分别为低 soc、高电机控制器温度、高电机转速、充电状态。其他诸多因素的相互作用同样对汽车故障的产生发生作用，基于本分析和前文分析，在下文针对行车充电、车辆自身故障检测和定期检修三个角度提出建议。

8.2 针对行车与充电的建议

针对**行车与充电对故障的影响**要考虑多个方面。考虑到车辆的**充电状态、电池健康**，以及相关系统的**工作状态**，可以优化充电策略，基于实时电池**电压、SOC、温度**等数据动态调整充电模式，避免**过度充电**对电池造成损害。监控电池**单体电压最高值、**

最低值等指标，通过实时数据分析及时发现异常，避免因电池故障导致的行车中断或事故。

建立**故障预警机制**，通过充电和行车过程中**电机控制器、驱动电机**等的**温度、转速**等数据，及时发现异常并通知驾驶员进行检查。最后，优化车辆状态管理，根据实时变化调整行车模式，比如充电时限制某些功能，延长电池寿命，避免电力不均衡。这样能够更精准地基于行车和充电过程中出现的实时数据做出调整，避免车辆在行驶过程中出现问题。

8.3 针对车辆自身故障监测的建议

针对该车辆的本身问题的检测要关注多个方面。首先，在动力电池的使用过程中，必须重点关注**温度、SOC 变化率、电压与电流比、单体电压波动**等关键指标。电池的**温度**和 **SOC 值**应始终保持在合理范围内，避免过度充放电以及电池单体电压不平衡等问题，这些都会加速电池衰减或引发故障。同时，建议定期检测电池的**电压与电流比**，确保电池的充放电效率稳定，防止因电池性能下降导致动力系统故障。

控制系统的运行状态也是不可忽视的，尤其是**电机的温度**与**转速**之间的关系。当电机温度升高，且与转速之间的斜率异常时，应及时检查电机系统，防止因过热而导致控制系统失效。电机**转矩的波动**也需要特别关注，过大的波动可能影响车辆的平稳运行，并对控制系统造成损害。因此，合理调节车辆的行驶负载，避免长时间高负荷运转，将有助于保障控制系统的稳定性和电机的健康。

在**充电系统**方面，电压与电流的匹配至关重要。过高或过低的充电功率都可能导致电池损坏，建议避免在低 SOC 状态下进行快速充电，并确保电池充电过程中的电流波动较小。对于频繁充电放电的车辆，应特别注意充电效率和充电功率，避免充电系统长时间处于高负荷状态。过度频繁的充电和深度放电不仅会加速电池老化，还可能引发其他系统故障。

车辆的**使用规律**也需要关注。长时间的超速行驶或高速运行可能导致电池和电机的过载运行，增加故障发生的风险。建议避免车辆长时间处于极限工况下运行。

综上所述，通过密切关注**电池温度、SOC、电压与电流比、电机状态、充电系统的运行状况**以及**用车规律**等关键指标，合理规划充电和使用习惯，能够有效降低故障发生的几率，延长车辆的使用寿命，并确保各系统的高效运转。

8.4 针对故障检修的建议

根据最初的结果，针对 **故障检修** 的建议应综合考虑多个系统的检测与维护。首先，**电池管理系统** 需要定期检查，尤其是在长时间使用后，确保 **SOC**、**单体电压** 和 **电池温度** 始终处于合理范围。对于出现电池 **过度放电** 或 **充电不均衡** 的情况，应及时检测并校准电池系统，防止进一步的性能下降或损坏。其次，针对 **驱动电机** 和 **电机控制器**，应定期监控其 **温度**、**转速** 和 **转矩**，特别是在高负荷运转时，如果电机温度异常升高，应进行检查并采取冷却或更换部件措施。此外，电机 **转矩波动** 过大的情况也需

要引起警觉，过大的波动可能会影响车辆的稳定性，进而引发故障。在 **充电系统** 方面，应对 **电压与电流比** 进行定期检测，确保充电系统正常工作，避免因过高或过低的充电功率对电池造成损害。最后，应建立系统的 **故障预警机制**，实时监控车辆的各项指标，及时发现异常并进行故障排查，确保车辆在行驶过程中的稳定性和安全性。

九、 总结

本文聚焦新能源汽车故障预警体系的构建与优化，形成了一个具有理论创新性与工程实用性的完整技术解决方案。

针对问题一，本文首先对附件数据进行了预处理操作，为后续的建模等操作梯控可靠的数据集支持，然后通过多种可视化方案对该车主的充电、用车规律以及该新能源车的故障规律进行总结与展示。

针对问题二，本文采取了一种通过多种方法联合进行特征筛选的策略，具体而言，首先通过多种方法构建了能够全面完整刻画车辆状态的四十种特征工程，然后基于此，通过互信息法、ANOVA 方差分析、随机森林特征重要性、聚类驱动特征筛选、物理模型特征验证五种方法分别进行特征筛选，最后通过交叉验证机制确定主要影响因素。

针对核心建模预测问题三和四，本文创新设计基于增强型双塔架构的深度学习模型和基于分层集成架构的深度学习模型两种模型来完成这一任务。其中双塔架构模型在 Val Acc 的关键指标上优于分层集成架构模型与传统的 CarBoost、XGboost、LightGBM 模型，成功解决了时序预测中"精度-可解释性"的权衡难题。并基于此模型完成了对问题四的预测任务。

最终根据前文所做工作构建了一个基于 Apriro 的关联分析模型，基于该模型分析结果和前文结论得出了：针对行车与充电方面：应当动态调整充电模式，避免过度充电对电池造成损害，根据实时变化调整行车模式等建议；针对车辆自身故障检测方面：重点关注温度、SOC 变化率、电压与电流比、单体电压波动等关键指标，电压与电流应相匹配等建议。

本研究的理论价值在于构建了"数据-机理-算法-策略"四位一体的故障预测预警方法论，为解决新能源汽车领域"海量数据低价值密度"与"复杂故障难溯源"的双重困境提供了创新路径。

十、 模型的评价、改进与推广

10.1 模型的优点

1.通过先进的机器学习方法和数据清洗工作，对现有数据集中的缺失值和异常值进行补充和修正

2.通过特征工程和物理模型特征验证结合电驱系统能量传递机理，人工构建反映电池热失控、电机过载等故障前兆的衍生特征（，增强模型的工程可解释性。

3.通过聚类方法进行特征筛选与 ANOVA 和互信息法等多维度特征归因分析方法，对各类故障进行分类，并得到各类故障的主要影响因素。

10.2 模型的不足

1.数据依赖性强：现有的数据集的质量直接影响到模型的预测能力。缺失值和异常值的补充虽然有助于提高模型的稳定性，但如果数据本身存在较大偏差或不完整，模型的性能仍可能不如预期。

2.特征工程的主观性：尽管通过结合物理模型特征来构建衍生特征可以增强模型的可解释性，但这些人工特征可能存在一定的主观性，且未必完全反映系统的复杂性，可能错过一些关键的未显性特征。

3.聚类方法的局限性：聚类方法在特征筛选过程中对样本的分组依赖较强，若数据集不均衡或者特征分布不规则，可能导致误分类，从而影响后续的故障分类精度。

10.3 模型的改进

1.增强数据集多样性与质量：通过数据增强技术（如数据合成、平衡采样等），提高数据的多样性和代表性，减少缺失值对模型的影响。同时，引入数据清洗算法，如异常检测算法（Isolation Forest 等）进行更精细化的数据预处理。

2.自动化特征选择和生成：利用自动化机器学习（AutoML）或深度学习技术（如特征自动生成器），减轻人工构建特征的工作量，且能够更全面、客观地发现潜在的、有助于故障预测的特征，提升模型性能。

3.更精细的聚类与分类方法：采用更先进的聚类技术（如 DBSCAN、层次聚类）来改进聚类过程，减少聚类算法对数据分布的依赖，同时引入深度学习模型（如自编码器）结合聚类进行特征降维，从而提升模型的分类精度。

10.4 模型的推广

1.跨领域应用：该模型不仅适用于电驱系统故障检测，还可以推广到其他设备或工业系统（如机械故障监测、生产设备预警系统）中，进行数据驱动的故障预警和优化维护。

2.集成模型与实时监控：结合在线学习和实时数据采集系统，在运行过程中实时更新模型，基于新的数据反馈进行自适应调整，从而使模型在实际应用中更加灵活和准确。

3.向边缘计算和物联网设备扩展：结合边缘计算和物联网（IoT）技术，将该模型部署到智能硬件设备中，使得设备能够自主进行故障预测和健康管理，从而实现更高效的故障检测和维护，提升系统的可靠性和可维护性。

十一、 参考文献

[1] 刘和平，刘平，胡银全（改善电动汽车动力性能的双向 Z 源逆变器控制策略[J]电工技术学报 27(2)，2012

[2] 高辉,彭成薇,李炜卓,李奕杰 &陈良亮.(2024).电动汽车与充电设备充电 安全预警研究综述.电力系统自动化(07),47-61.

[3] 任超.(2023).基于数据驱动的新能源汽车动力电池故障诊断与预警研究 (硕士学位论文,长安大学).硕士

[4] 李钰.(2024).新能源汽车维修与故障诊断技术研究.中国设备工程 (15),183-185.

[5] 郭忠军.基于小波包分解与样本熵的新能源汽车变速器故障诊断研究[J].汽车维修技师,2025,(04):39-40.

[6] 倪龙飞,白倩,张治斌.基于 AdaBoost 算法的新能源汽车电机异常故障检测[J].计算机仿真,2024,41(04):97-101.

[7] 傅振亮,张建敏,罗崴,等.电池充电状态和电压/温度异常联合故障诊断[J].科学技术与工程,2024,24(08):3260-3268.

[8] 刘丹,王瑞虎,吕伟,等.基于 IPSO-LSTM 的新能源汽车锂电池健康状态监测[J].中国安全科学学报,2023,33(09):94-102.DOI:10.16265/j.cnki.issn1003-3033.2023.09.1250.

[9] 胡健,周俊,刘韬,等.主密度峰值聚类在 RV 减速器复合故障的应用

[10] 振动工程学报,1-10[2025-03-16.http://kns.cnki.net/kcms/detail/32.1349.TB.20250219.1706.006.html.

[11] Yujin Tang, Lu Qi,Fei Xie, Xiangtai Li ,Chao. MaPREDFORMER: TRANSFORMERS ARE EFFECTIVESPATIAL-TEMPORAL PREDICTIVE LEARNERS[J]. Applied Energy, 343, 121789.

[12] 赵景远、迅宁凤、俊王斌、连玉波、明高.利用时空变压器网络的电动汽车蓄电池故障诊断及故障预测.

[13] 李鑫颜.解读《电动汽车科技发展“十二五”专项规划(摘要)》新能源客车任重而道远[J].商用汽车新闻,2012,(11):2.

[14] 马铖,赵晓龙,鲁亚云,等.基于电子诊断技术的新能源汽车检测与维修技术研究[J].汽车零部件,2024,(02):116-119.DOI:10.19466/j.cnki.1674-1986.2024.2.022.

[15] 杨青,刘军.基于 TDOA 数据采集和传输效率的优化方法[J].信息记录材料,2023,24(05):210-212.DOI:10.16009/j.cnki.cn13-1295/tq.2023.05.020.

[16] 王玉龙,蔡志端,徐静云,等.串联锂离子电池组电压巡检电路[J].电子设计工程,2019,27(13):143-147.DOI:10.14022/j.cnki.dzsjgc.2019.13.031.

[17] 邓吉利,曹太强,陈汝兵,等.蓄电池充电电流控制的 LLC 谐振变换器研究[J].电测与仪表,2021,58(04):73-77+176.DOI:10.19753/j.issn1001-1390.2021.04.011.

[18] 杨建飞,胡育文.永磁同步电机直接转矩控制转矩调节器设计[J].中国电机工程学报,2011,31(09):76-81.DOI:10.13334/j.0258-8013.pcsee.2011.09.012.
[19] 甘志恒.基于机器学习的纯电动汽车 SOC 估算方法研究[D].华南理工大学,2022.DOI:10.27151/d.cnki.ghnlu.2022.002217.
[20] 蒋大治,张亮亮.基于变异系数的交通流序列突变检测方法[J].智能城市,2017,3(10):38-40.DOI:10.19301/j.cnki.zncs.2017.10.013.
[21] Liu X ,Wang W .Deep Time Series Forecasting Models: A Comprehensive Survey[J].Mathematics,2024,12(10):
[22] Rong Z ,Sun W ,Xie Y , et al.Mixture of experts leveraging Informer and LSTM variants for enhanced daily streamflow forecasting[J].Journal of Hydrology,2025,653132737-132737.

十二、 附录

代码名称	代码解决的问题	代码电子版所在的位置
文件合并.py	第一题数据合并	支撑材料文件夹
数据预处理.py	第一题数据处理	支撑材料文件夹
随机森林填充.py	第一题异常值补充	支撑材料文件夹
每天里程折线图.py	第一题数据可视化	支撑材料文件夹
每天平均里程折线图.py	第一题数据可视化	支撑材料文件夹
每月充电与 SOC 图.py	第一题数据可视化	支撑材料文件夹
每月里程.py	第一题数据可视化	支撑材料文件夹
每月每小时故障图.py	第一题数据可视化	支撑材料文件夹
每月每小时用车图.py	第一题数据可视化	支撑材料文件夹
每周每小时充电热力图.py	第一题数据可视化	支撑材料文件夹
每周每小时故障热力图.py	第一题数据可视化	支撑材料文件夹
每周每小时用车图.py	第一题数据可视化	支撑材料文件夹
每周每小时用车图 2.py	第一题数据可视化	支撑材料文件夹
每周每月每小时充电雷达图.py	第一题数据可视化	支撑材料文件夹
每周每月每小时故障雷达图.py	第一题数据可视化	支撑材料文件夹
每周每月每小时用车面积图.py	第一题数据可视化	支撑材料文件夹
用车与故障交叉谱密度图.py	第一题数据可视化	支撑材料文件夹
问题二.py	第二题代码	支撑材料文件夹
问题三训练.py	第三题代码	支撑材料文件夹
问题三 CatBoost 稳健性检验.py	第三题稳健性检验	支撑材料文件夹
问题三LightGBM稳健性检验.py	第三题稳健性检验	支撑材料文件夹
问题三 SVC 稳健性检验.py	第三题稳健性检验	支撑材料文件夹
问题三 XGboost 稳健性检验.py	第三题稳健性检验	支撑材料文件夹
问题三随机森林稳健性检验.py	第三题稳健性检验	支撑材料文件夹
		支撑材料文件夹

附录 1
介绍：文件合并.py，该代码由 python 编写

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor

# 需要填补的列（目标变量）
target_columns = [
    "Motor Controller Temperature", "Motor Speed", "Motor Torque",
    "Motor Temperature", "Motor Controller Input Voltage", "Motor Controller DC
Bus Current"
]

# 预测特征（加入 Total Voltage）
feature_columns = ["Max Battery Voltage", "Min Battery Voltage", "Total Voltage"]

# 确保数据中包含所有列
valid_target_columns = [col for col in target_columns if col in final_data.columns]
valid_feature_columns = [col for col in feature_columns if col in
final_data.columns]

# 转换预测特征数据类型，确保都是数值型
final_data[valid_feature_columns] =
final_data[valid_feature_columns].astype(float)

# 逐列填补缺失值（只对缺失目标变量进行预测填补，保留其他缺失值行）
for target_col in valid_target_columns:
    print(f"正在填补列: {target_col}")

    # 选择没有缺失目标列及预测特征的行作为训练数据
    df_train = final_data.dropna(subset=[target_col] + valid_feature_columns)
    # 选择缺失目标列的行（预测数据）；如果预测特征中有缺失也不参与预
测
    df_missing = final_data[final_data[target_col].isna() &
final_data[valid_feature_columns].notna().all(axis=1)]

    print(f"训练样本数: {df_train.shape[0]}，需要填补的样本数:
{df_missing.shape[0]}")

    if not df_train.empty and not df_missing.empty:
        X_train = df_train[valid_feature_columns]
        y_train = df_train[target_col]

        model = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
            model.fit(X_train, y_train)

            X_missing = df_missing[valid_feature_columns]
            print("预测前预测特征的缺失统计：")
            print(X_missing.isna().sum())

            predicted_values = model.predict(X_missing)
            final_data.loc[final_data[target_col].isna(), target_col] = predicted_values
            print(f"填补完成: {target_col}")

    print("所有目标列的缺失值填补完成！")

    # 保存填补后的数据
    output_file = os.path.join(data_dir, "merged_cleaned_data_filled.csv")
    final_data.to_csv(output_file, index=False, encoding="utf-8")
    print(f"填补后的数据已保存至: {output_file}")

    import os
    print("当前工作目录:", os.getcwd())
```

附录 2

介绍：数据预处理.py，该代码由 python 编写

```
    import pandas as pd
    import numpy as np
    from sklearn.feature_selection import mutual_info_classif, f_classif
    from sklearn.preprocessing import StandardScaler
    from sklearn.cluster import KMeans
    from sklearn.metrics import silhouette_score
    from sklearn.ensemble import RandomForestClassifier
    import matplotlib.pyplot as plt
    import seaborn as sns
    from collections import defaultdict
    from sklearn.cluster import MiniBatchKMeans
    from sklearn.feature_selection import mutual_info_classif, f_classif
    from sklearn.ensemble import RandomForestClassifier

    # Load data
    df = pd.read_csv(r'C:\Users\黄新页
\Downloads\74f7c310f933fd8a38414cc893a12e371741787112770\2024 年第四届高校大
数据挑战赛赛题\2024 年第四届高校大数据挑战赛-B 赛题\附件\month_01.csv',
parse_dates=[0], encoding='gbk')
    #df = pd.read_excel(r'D:\month_09.xlsx', parse_dates=[0])
    # Set column names as specified
    df.columns = [
```

```
        '数据采集时间', '车辆状态', '充电状态', '车速', '累计里程', '总电压', '总电流',
'SOC',
        'DC-DC 状态', '绝缘电阻', '驱动电机控制', '驱动电机转速', '驱动电机转矩',
        '驱动电机温度', '电机控制器输入电压', '电机控制器直流母线电流', '电池单
体电压最高值',
        '电池单体电压最低值', '最高温度值', '最低温度值','最高报警等级'
    ]


    # 预处理增强
    def enhanced_preprocessing(df):
        # 时间排序
        df = df.sort_values('数据采集时间')

        # 缺失值处理
        print("缺失值统计（预处理前）:")
        print(df.isnull().sum())

        # 分类型数据填充
        categorical_cols = ['车辆状态', '充电状态', 'DC-DC 状态']
        for col in categorical_cols:
            if col in df.columns:
                df[col] = df[col].ffill().bfill()

        # 数值型数据填充
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        for col in numeric_cols:
            if col != '数据采集时间':
                # 使用移动窗口均值填充
                df[col] = df[col].fillna(df[col].rolling(10, min_periods=1).mean())
                df[col] = df[col].fillna(method='ffill').fillna(method='bfill')

        print("缺失值统计（预处理后）:")
        print(df.isnull().sum())
        return df


    df = enhanced_preprocessing(df)


    # 改进的数据清洗
    def enhanced_clean_numeric(x):
        try:
            if isinstance(x, str):
```

```
                x = x.strip().replace(' ', '')
                if ':' in x:   # 处理时间格式
                    parts = list(map(float, x.split(':')))
                    return parts[0] + parts[1] / 60
                return float(x)
            return float(x)
        except:
            return np.nan


numeric_cols = df.select_dtypes(include=[np.number, object]).columns.tolist()
for col in numeric_cols:
    if col != '数据采集时间':
        df[col] = df[col].apply(enhanced_clean_numeric)
        df[col] = pd.to_numeric(df[col], errors='coerce')


# 改进的异常值处理
def improved_handle_outliers(df, column):
    if df[column].nunique() < 5:
        return df   # 忽略类别型特征

    # 使用改进的分位数计算
    Q1 = df[column].quantile(0.05)
    Q3 = df[column].quantile(0.95)
    IQR = Q3 - Q1

    if IQR == 0:
        # 使用标准差法作为备用方案
        mean = df[column].mean()
        std = df[column].std()
        lower_bound = mean - 3 * std
        upper_bound = mean + 3 * std
    else:
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

    # 使用中位数填充异常值
    df[column] = np.where(
        (df[column] < lower_bound) | (df[column] > upper_bound),
        df[column].median(),
        df[column]
    )
    return df
```

```
    for col in numeric_cols:
        if col not in ['数据采集时间', '最高报警等级']:
            df = improved_handle_outliers(df, col)

    # Feature engineering
    epsilon = 1e-3
    df['时间差'] = df['数据采集时间'].diff().dt.total_seconds() / 3600


    df['电压差异'] = df['电池单体电压最高值'] - df['电池单体电压最低值']
    df['温度差异'] = df['最高温度值'] - df['最低温度值']
    df['SOC 变化率'] = df['SOC'].diff() / (df['时间差'] + epsilon)
    df['温度变化率'] = df['最高温度值'].diff() / (df['时间差'] + epsilon)
    df['功率输出'] = df['总电压'] * df['总电流']
    df['电机功率'] = df['电机控制器输入电压'] * df['电机控制器直流母线电流']
    df['能量效率'] = df['功率输出'] / (df['车速'] + epsilon)
    df['充电电流'] = np.where(df['充电状态'] == 1, df['总电流'], 0)
    df['充电时长'] = df['时间差'].where(df['充电状态'] == 1, 0).cumsum()
    df['时间段'] = np.where((df['数据采集时间'].dt.hour >= 6) & (df['数据采集时间
'].dt.hour < 18), 1, 0)
    df['电机转矩-转速比'] = df['驱动电机转矩'] / (df['驱动电机转速'] + epsilon)
    df['电压-电流比'] = df['总电压'] / (df['总电流'] + epsilon)
    df['电机温度-转速斜率'] = df['驱动电机温度'].diff() / (df['驱动电机转速'].diff() +
epsilon)
    df['转矩波动'] = df['驱动电机转矩'].rolling(window=10, min_periods=1).std()
    df['效率'] = df['电机功率'] / (df['电机控制器输入电压'] * df['电机控制器直流母线
电流'] + epsilon)
    df['车速变化率'] = df['车速'].diff() / (df['时间差'] + epsilon)
    df['急变频率'] = df['车速变化率'].rolling(window=10).apply(lambda x:
np.sum(np.abs(x) > 2), raw=False)
    df['能耗率'] = df['累计里程'].diff() / (df['SOC'].diff() + epsilon)
    df['前兆特征'] = df['温度变化率'].rolling(window=10).mean()
    df['相关系数'] = df['最高温度值'].rolling(window=10).corr(df['驱动电机温度'])
    df['高速 SOC 下降率'] = df['SOC'].diff() / (df['时间差'] + epsilon) * (df['车速'] > 80)
    df['耦合指标'] = df['总电流'] * df['最高温度值']
    df['急加速频率'] = (df['车速变化率'] > 3).rolling(10).sum()
    df['急刹车频率'] = (df['车速变化率'] < -3).rolling(10).sum()
    df['超速运行时长'] = df['时间差'].where(df['车速'] > 100, 0).cumsum()
    df['频繁充放电次数'] = (df['充电状态'].diff() != 0).cumsum()
    df['低 SOC 运行时长'] = df['时间差'].where(df['SOC'] < 20, 0).cumsum()

    df['电机高负荷运行时长'] = df['时间差'].where(
```

```
            (df['驱动电机转速'] > 0.8 * df['驱动电机转速'].max()) |
            (df['驱动电机转矩'] > 0.8 * df['驱动电机转矩'].max()), 0
        ).cumsum()
        df['充电速率'] = df['总电流'].where(df['充电状态'] ==
1).rolling(window=10).mean()
        # 1. 电池系统特征
        df['电池压差变化率'] = df['电池单体电压最高值'].diff() - df['电池单体电压最低值
'].diff()
        df['最大单体电压波动'] = df['电池单体电压最高值'].rolling(60).std()
        df['电压一致性指数'] = df['电池单体电压最高值'] / (df['电池单体电压最低值'] +
epsilon)

        # 2. 充电特征增强
        df['充电功率'] = df['总电压'] * df['总电流'].where(df['充电状态'].isin([1, 2]))
        df['充电效率'] = df['SOC'].diff() / (df['充电功率'] + epsilon)

        # 3. 电机系统特征
        df['电机综合负载率'] = (df['驱动电机转速'] / 10000 + df['驱动电机转矩'] / df['驱动
电机转矩'].max()) / 2
        df['电机热负荷'] = df['驱动电机温度'] * df['驱动电机转速'] / 1000

        # 4. 温度动态特征
        df['温升加速度'] = df['最高温度值'].diff().diff() / (epsilon + df['时间差'].diff())
        df['温度梯度'] = (df['最高温度值'] - df['最低温度值']) / df['绝缘电阻']

        # 5. 时间序列特征
        window_config = {
            '1min': 60,
            '10min': 600,
            '1hour': 3600
        }

        for col in ['总电流', 'SOC', '绝缘电阻']:
            for name, size in window_config.items():
                df[f'{col}_{name}_变异系数'] = df[col].rolling(size).std() /
(df[col].rolling(size).mean() + epsilon)

        # Fill NaN values

        df = df.ffill().bfill()



        # Feature selection
```

```
    features = [
        'SOC', '总电压', '总电流', '电池单体电压最高值', '电池单体电压最低值', '最高温度值', '最低温度值',
        '驱动电机转速', '驱动电机温度', '车速', '累计里程', '充电状态', '绝缘电阻', '电压差异',
        '温度差异', 'SOC 变化率', '温度变化率', '功率输出', '电机功率', '能量效率', '充电电流',
        '充电时长', '时间段', '电机转矩-转速比', '电压-电流比', '电机温度-转速斜率', '转矩波动',
        '效率', '急变频率', '能耗率', '前兆特征', '相关系数', '高速 SOC 下降率', '耦合指标',
        '急加速频率', '急刹车频率', '超速运行时长', '频繁充放电次数', '低 SOC 运行时长',
        '电机高负荷运行时长', '充电速率', '电池压差变化率', '最大单体电压波动', '电压一致性指数',
        '充电功率', '充电效率', '电机综合负载率', '电机热负荷',
        '温升加速度', '温度梯度'
    ]



    # Ensure target column exists
    if '最高报警等级' not in df.columns:
        df['最高报警等级'] = pd.qcut(df['SOC'], q=4, labels=False, duplicates='drop') # Example fallback
        print("警告：'最高报警等级'列不存在，使用 SOC 分位数生成示例数据")

    # Check data distribution across alarm levels
    print("各报警等级样本数：")
    print(df['最高报警等级'].value_counts())

    # Prepare feature matrix
    X = df[features].astype(float).dropna()
    y = df['最高报警等级'].astype(int).loc[X.index]
    X.replace([np.inf, -np.inf], np.nan, inplace=True)
    X = X.fillna(X.median())
    # Feature selection for each alarm level (0, 1, 2, 3)
    results_by_level = {level: {} for level in range(4)}

    for level in range(4):
        print(f"\n 处理报警等级 {level} 的特征归因分析")
        # Create binary target: 1 for current level, 0 for others
        y_binary = (y == level).astype(int)
```

```
        # Mutual Information
        mi_scores = mutual_info_classif(X, y_binary, random_state=42)
        mi_df = pd.DataFrame({'特征': features, '互信息得分':
mi_scores}).sort_values('互信息得分', ascending=False)
        results_by_level[level]['互信息法'] = mi_df.head(5)['特征'].tolist()

        # ANOVA F-test
        f_scores, p_values = f_classif(X, y_binary)
        anova_df = pd.DataFrame({'特征': features, 'F 值': f_scores, 'P 值':
p_values}).sort_values('F 值', ascending=False)
        results_by_level[level]['ANOVA 检验'] = anova_df.head(5)['特征'].tolist()

        # Random Forest
        rf = RandomForestClassifier(n_estimators=25, random_state=42, n_jobs=-1)
        rf.fit(X, y_binary)
        rf_importance = pd.DataFrame({'特征': features, '重要性':
rf.feature_importances_}).sort_values('重要性',

ascending=False)
        results_by_level[level]['随机森林'] = rf_importance.head(5)['特征'].tolist()

        # Clustering (optional, kept for consistency but less relevant for per-level
analysis)
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)
        silhouette_scores = [silhouette_score(X_scaled, KMeans(n_clusters=k,
random_state=42).fit_predict(X_scaled))
                                    for k in range(2, 6)]
        optimal_k = range(2, 6)[np.argmax(silhouette_scores)]
        kmeans = KMeans(n_clusters=optimal_k, random_state=42)
        cluster_labels = kmeans.fit_predict(X_scaled)
        df.loc[X.index, f'聚类标签_level_{level}'] = cluster_labels
        cluster_means = df.groupby(f'聚类标签_level_{level}')[features].mean()
        cluster_diff =
cluster_means.sub(cluster_means.mean()).abs().sum(axis=0).sort_values(ascending=False)
        cluster_features = pd.DataFrame({'特征': cluster_diff.index, '得分':
cluster_diff.values})
        results_by_level[level]['聚类分析'] = cluster_features.head(5)['特征'].tolist()

        # Physical model features (example, adjust as needed)
        results_by_level[level]['物理模型'] = ['SOC_EKF', 'R0_EKF', '最高温度值']

    # Summarize results for each level
    for level in range(4):
```

```
        print(f"\n 报警等级 {level} 的主要影响因素：")
        feature_counts = defaultdict(int)
        for method, feats in results_by_level[level].items():
            for feat in feats:
                feature_counts[feat] += 1
        top_features = sorted(feature_counts.items(), key=lambda x: x[1],
reverse=True)[:10]
        for feat, count in top_features:
            print(f"{feat} （在{count}种方法中出现）")

    # Visualization: One plot per method, showing all levels
    fig, axes = plt.subplots(2, 2, figsize=(20, 15))
    fig.suptitle("各报警等级特征重要性对比", fontsize=16)

    methods = ['互信息法', 'ANOVA 检验', '聚类分析', '随机森林']
    axes_flat = axes.flatten()

    for idx, method in enumerate(methods):
        ax = axes_flat[idx]
        top_features_all = []
        for level in range(4):
            top_features_all.extend(results_by_level[level][method])
        top_features_unique = list(dict.fromkeys(top_features_all))[:7]   # Top 5 unique
features
        scores = {feat: [0] * 4 for feat in top_features_unique}
        for level in range(4):
            importance_df = (
                pd.DataFrame({
                    '特征': features,
                    '得分': mutual_info_classif(X, (y == level).astype(int),
random_state=42)
                }) if method == '互信息法' else
                pd.DataFrame({
                    '特征': features,
                    '得分': f_classif(X, (y == level).astype(int))[0]
                }) if method == 'ANOVA 检验' else
                pd.DataFrame({
                    '特征': features,
                    '得分': RandomForestClassifier(random_state=42)
                    .fit(X, (y == level).astype(int)).feature_importances_
                }) if method == '随机森林' else
                cluster_features
            )
```

```
                for feat in top_features_unique:
                    scores[feat][level] = (importance_df[importance_df['特征'] == feat]['
得分'].values[0]
                                                    if feat in importance_df['特征'].values
else 0)

        # 创建单个图表而不是多个
        fig, ax = plt.subplots(figsize=(10, 6))
        bar_width = 0.15
        x = np.arange(len(top_features_unique))

        # 绘制所有级别的条形图
        for level in range(4):
            ax.bar(x + level * bar_width,
                   [scores[feat][level] for feat in top_features_unique],
                   bar_width,
                   label=f'等级 {level}')

        plt.rcParams['font.sans-serif'] = ['SimHei']  # 正常显示中文标签
        plt.rcParams['axes.unicode_minus'] = False
        ax.set_title(method)
        ax.set_xticks(x + bar_width * 1.5)
        ax.set_xticklabels(top_features_unique, rotation=45)
        ax.legend()
        plt.tight_layout(rect=[0, 0, 1, 0.95])
        plt.show()
        ax.set_title(method)
        ax.set_xticks(x + bar_width * 1.5)
        ax.set_xticklabels(top_features_unique, rotation=45)
        ax.legend()
        plt.tight_layout(rect=[0, 0, 1, 0.95])
        plt.show()
```

附录 3

介绍：随机森林填充.py，该代码由 python 编写

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor

# 需要填补的列（目标变量）
target_columns = [
    "Motor Controller Temperature", "Motor Speed", "Motor Torque",
    "Motor Temperature", "Motor Controller Input Voltage", "Motor Controller DC
Bus Current"
]
```

```
    # 预测特征（加入 Total Voltage）
    feature_columns = ["Max Battery Voltage", "Min Battery Voltage", "Total Voltage"]

    # 确保数据中包含所有列
    valid_target_columns = [col for col in target_columns if col in final_data.columns]
    valid_feature_columns = [col for col in feature_columns if col in
final_data.columns]

    # 转换预测特征数据类型，确保都是数值型
    final_data[valid_feature_columns] =
final_data[valid_feature_columns].astype(float)

    # 逐列填补缺失值（只对缺失目标变量进行预测填补，保留其他缺失值行）
    for target_col in valid_target_columns:
        print(f"正在填补列: {target_col}")

        # 选择没有缺失目标列及预测特征的行作为训练数据
        df_train = final_data.dropna(subset=[target_col] + valid_feature_columns)
        # 选择缺失目标列的行（预测数据）；如果预测特征中有缺失也不参与预
测
        df_missing = final_data[final_data[target_col].isna() &
final_data[valid_feature_columns].notna().all(axis=1)]

        print(f"训练样本数: {df_train.shape[0]}，需要填补的样本数:
{df_missing.shape[0]}")

        if not df_train.empty and not df_missing.empty:
            X_train = df_train[valid_feature_columns]
            y_train = df_train[target_col]

            model = RandomForestRegressor(n_estimators=100, random_state=42)
            model.fit(X_train, y_train)

            X_missing = df_missing[valid_feature_columns]
            print("预测前预测特征的缺失统计：")
            print(X_missing.isna().sum())

            predicted_values = model.predict(X_missing)
            final_data.loc[final_data[target_col].isna(), target_col] = predicted_values
            print(f"填补完成: {target_col}")

    print("所有目标列的缺失值填补完成！")
```

```
# 保存填补后的数据
output_file = os.path.join(data_dir, "merged_cleaned_data_filled.csv")
final_data.to_csv(output_file, index=False, encoding="utf-8")
print(f"填补后的数据已保存至: {output_file}")

import os
print("当前工作目录:", os.getcwd())
```

附录 4

介绍：第五题.ipynb，该代码由 python 编写

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# 假设 all_data 已经加载为 pandas DataFrame
df = all_data.copy()

# 检查列名以确保没有拼写错误
print(df.columns)

# 确保相关列的类型为数值型
df['SOC'] = pd.to_numeric(df['SOC'], errors='coerce')  # 更新列名为 'SOC'
df['Motor Controller Temperature'] = pd.to_numeric(df['Motor Controller
Temperature'], errors='coerce')
df['Motor Temperature'] = pd.to_numeric(df['Motor Temperature'], errors='coerce')
df['Motor Speed'] = pd.to_numeric(df['Motor Speed'], errors='coerce')

# 数据预处理：将某些数值型数据转换为二进制数据，表示特定的条件

# 示例: SOC 小于 20%时为 1，表示低电量状态
df['Low_SOC'] = (df['SOC'] < 20).astype(int)

# 示例: 电池控制器温度大于 50 度时为 1，表示温度过高
df['High_Motor_Controller_Temperature'] = (df['Motor Controller Temperature'] >
50).astype(int)

# 示例: 电机温度大于 80 度时为 1，表示电机温度过高
df['High_Motor_Temperature'] = (df['Motor Temperature'] > 80).astype(int)

# 示例: 电机转速大于 3000rpm 时为 1，表示高转速
df['High_Motor_Speed'] = (df['Motor Speed'] > 3000).astype(int)

# 示例: 充电状态为 1，表示正在充电
```

```
df['Charging'] = (df['Charging Status'] == 1).astype(int)  # 更新列名为 'Charging Status'

# 示例: 车辆状态是否异常（如故障）
df['Vehicle_Fault'] = (df['Vehicle Status'] == 'Fault').astype(int)

# 选择进行关联分析的特征，包含故障发生
df_association = df[['Low_SOC', 'High_Motor_Controller_Temperature', 'High_Motor_Temperature',
                     'High_Motor_Speed', 'Charging', 'Vehicle_Fault']]

# 检查 df_association 是否为空
print(df_association.head())  # 查看前几行数据
print(df_association.describe())  # 查看数据的统计信息

# 如果发现缺失值，删除缺失值
df_association = df_association.dropna()

# 使用 Aprior 算法挖掘频繁项集，设置最小支持度为 0.1（可以适当降低）
frequent_itemsets = apriori(df_association, min_support=0.01, use_colnames=True)

# 如果没有频繁项集，检查是否支持度过高
if frequent_itemsets.empty:
    print("没有找到频繁项集，请检查数据和支持度值")
else:
    # 生成关联规则，设置最小置信度为 0.7
    rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
    print('BBB')
    # 输出所有关联规则
    print(rules)

    # 筛选与故障相关的规则（查看故障发生的相关规则）
    print('AAA')
    interesting_rules = rules[rules['consequents'].apply(lambda x: 'Vehicle_Fault' in x)]
    print(interesting_rules)
```

附录 5

介绍：集成学习.py，该代码由 python 编写

```
import pandas as pd
import numpy as np
import warnings
from sklearn.preprocessing import StandardScaler
```

```
from tensorflow.keras import models, layers
from xgboost import XGBClassifier

warnings.filterwarnings('ignore')

WINDOW_SIZE = 12
ALARM_LEVELS = 4
BATCH_SIZE = 512
EPOCHS = 15


def preprocess_data(df, window_size):
    time_diff = pd.to_datetime(df['数据采集时间']).diff().dt.total_seconds().values
/ 3600
    time_diff[0] = 0.0
    required_columns = ['总电压', '总电流', 'SOC', '绝缘电阻', '电池单体电压最
高值', '电池单体电压最低值']
    if not all(col in df.columns for col in required_columns):
        raise ValueError("数据中缺少必要的特征列！")
    values = df[required_columns].values
    voltage_diff = values[:, 4] - values[:, 5]
    soc_rate = np.diff(values[:, 2], prepend=0) / np.where(time_diff < 1e-5, 1e-5,
time_diff)
    voltage_mean = pd.Series(values[:, 0]).rolling(window=5,
min_periods=1).mean().values
    insulation_std = pd.Series(values[:, 3]).rolling(window=3,
min_periods=1).std().values
    features = np.column_stack([
        values, voltage_diff.reshape(-1, 1), soc_rate.reshape(-1, 1),
        voltage_mean.reshape(-1, 1), insulation_std.reshape(-1, 1),
        np.log1p(np.abs(values[:, 1])).reshape(-1, 1), time_diff.reshape(-1, 1)
    ])
    features = np.nan_to_num(features, nan=0.0)
    scaler = StandardScaler()
    scaled_features = scaler.fit_transform(features)
    n_samples = len(scaled_features) - window_size + 1
    X = np.array([scaled_features[i:i + window_size] for i in range(n_samples)])
    y = df['最高报警等级'].values[window_size - 1:]
    return X, y, scaler


def create_cpu_model(input_shape):
    model = models.Sequential([
        layers.Conv1D(8, 2, activation='relu', input_shape=input_shape),
```

```
                layers.MaxPooling1D(2),
                layers.Flatten(),
                layers.Dense(16, activation='relu'),
                layers.Dropout(0.2),
                layers.Dense(ALARM_LEVELS, activation='softmax')
            ])
            model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
            return model


    def cpu_training(X_train, y_train):
            model = create_cpu_model((WINDOW_SIZE, X_train.shape[2]))
            model.fit(
                X_train, y_train,
                epochs=EPOCHS,
                batch_size=BATCH_SIZE,
                validation_split=0.1,
                class_weight={0: 1, 1: 5, 2: 20, 3: 50},
                verbose=1
            )
            xgb = XGBClassifier(n_estimators=200, max_depth=5, learning_rate=0.1,
tree_method='hist',
                                    n_jobs=-1, subsample=0.8,
scale_pos_weight=sum(y_train == 0) / sum(y_train > 0))
            xgb.fit(X_train.reshape(X_train.shape[0], -1), y_train)
            return {'CNN': model, 'XGB': xgb}


    def cpu_predict_and_update(models, scaler, path):
            df = pd.read_csv(path, encoding='gbk')
            time_diff = pd.to_datetime(df['数据采集时间']).diff().dt.total_seconds().values
/ 3600
            time_diff[0] = 0.0
            required_columns = ['总电压', '总电流', 'SOC', '绝缘电阻', '电池单体电压最
高值', '电池单体电压最低值']
            if not all(col in df.columns for col in required_columns):
                raise ValueError("预测数据缺少必要特征列！")
            values = df[required_columns].values
            voltage_diff = values[:, 4] - values[:, 5]
            soc_rate = np.diff(values[:, 2], prepend=0) / np.where(time_diff < 1e-5, 1e-5,
time_diff)
            voltage_mean = pd.Series(values[:, 0]).rolling(window=5,
min_periods=1).mean().values
```

```
        insulation_std = pd.Series(values[:, 3]).rolling(window=3,
min_periods=1).std().values
        features = np.column_stack([
            values, voltage_diff.reshape(-1, 1), soc_rate.reshape(-1, 1),
            voltage_mean.reshape(-1, 1), insulation_std.reshape(-1, 1),
            np.log1p(np.abs(values[:, 1])).reshape(-1, 1), time_diff.reshape(-1, 1)
        ])
        features = np.nan_to_num(features, nan=0.0)
        X_scaled = scaler.transform(features)
        seqs_pred = np.lib.stride_tricks.sliding_window_view(X_scaled,
WINDOW_SIZE, axis=0)
        cnn_pred = models['CNN'].predict(seqs_pred).argmax(1)
        xgb_pred = models['XGB'].predict(seqs_pred.reshape(len(seqs_pred), -1))
        final_pred = np.maximum(cnn_pred, xgb_pred)

        # 将预测结果插入原始数据框
        df.loc[WINDOW_SIZE - 1:, '最高报警等级'] = final_pred
        return df


    if __name__ == "__main__":
        combined_data = pd.DataFrame()
        base_path = r'C:\Users\黄新页
\Downloads\74f7c310f933fd8a38414cc893a12e371741787112770\2024 年第四届高校
大数据挑战赛赛题\2024 年第四届高校大数据挑战赛-B 赛题\附件'
        for i in range(1, 10):
            file_path = fr'{base_path}\month_0{i}.csv'
            for chunk in pd.read_csv(file_path, parse_dates=[0], encoding='gbk',
chunksize=10000):
                combined_data = pd.concat([combined_data, chunk],
ignore_index=True)

        full_X, full_y, scaler = preprocess_data(combined_data, WINDOW_SIZE)
        split_idx = int(0.9 * len(full_X))
        X_train, X_test = full_X[:split_idx], full_X[split_idx:]
        y_train, y_test = full_y[:split_idx], full_y[split_idx:]

        models = cpu_training(X_train, y_train)
        y_pred_cnn = models['CNN'].predict(X_test).argmax(1)
        y_pred_xgb = models['XGB'].predict(X_test.reshape(X_test.shape[0], -1))
        print("CNN 准确率:", (y_pred_cnn == y_test).mean())
        print("XGB 准确率:", (y_pred_xgb == y_test).mean())

        # 预测并更新 month_10.csv
```

```
        month_10_path = fr'{base_path}\month_10.csv'
        updated_df = cpu_predict_and_update(models, scaler, month_10_path)
        updated_df.to_csv(month_10_path, index=False, encoding='gbk')
        print(f'预测结果已插入并保存至 {month_10_path}")
```

附录 6

介绍：Dual-Tower PredFormer.py
，该代码由 python 编写

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.distributed as dist
from torch.utils.data import Dataset, DataLoader, DistributedSampler
from torch.nn.parallel import DistributedDataParallel as DDP
from sklearn.preprocessing import StandardScaler
import os
import time


class EVDataset(Dataset):
    def __init__(self, data, window_size=24):
        self.data = data
        self.window_size = window_size

    def __len__(self):
        return len(self.data) - self.window_size + 1

    def __getitem__(self, idx):
        window = self.data[idx:idx + self.window_size]
        x = window[:, :-1]
        y = window[-1, -1]
        alarm_level = int(y)
        return (
            torch.FloatTensor(x),
            torch.tensor(y, dtype=torch.long),
            torch.tensor(alarm_level, dtype=torch.long)
        )


class FeatureAwareTransformer(nn.Module):
    def __init__(self, dim, num_heads, alarm_feature_map, max_seq_len=24):
        super().__init__()
```

```
        self.attention = nn.MultiheadAttention(dim, num_heads)
        self.position_embed = nn.Parameter(torch.randn(max_seq_len, 1, dim))
# 位置编码

        # 特征掩码矩阵
        max_level = max(alarm_feature_map.keys())
        feature_masks = torch.zeros(max_level + 2, dim)
        for lvl, indices in alarm_feature_map.items():
            feature_masks[lvl, indices] = 1.0
        self.register_buffer('feature_masks', feature_masks)

    def forward(self, x, alarm_levels):
        # 添加位置编码
        x = x + self.position_embed[:x.size(0)]

        # 特征感知增强
        batch_size = x.size(1)
        masks = self.feature_masks[alarm_levels.long()]
        masks = masks.unsqueeze(0).expand(x.size(0), -1, -1)
        x = x * (masks + 0.5)   # 调整增强强度

        # 注意力计算
        attn_output, _ = self.attention(x, x, x)
        return attn_output


class EnhancedDualTower(nn.Module):
    def __init__(self, input_dim, window_size, num_classes, alarm_feature_map,
                 temporal_dim=128, channel_dim=128, num_heads=8):
        super().__init__()
        self.window_size = window_size

        self.temporal_encoder = nn.Sequential(
            nn.Linear(input_dim, temporal_dim),
            nn.GELU()
        )

        self.channel_encoder = nn.Sequential(
            nn.Linear(window_size, channel_dim),
            nn.GELU()
        )

        self.feature_aware = FeatureAwareTransformer(
            dim=temporal_dim,
```

```
                num_heads=num_heads,
                alarm_feature_map=alarm_feature_map
            )

            self.gate = nn.Sequential(
                nn.Linear(temporal_dim + channel_dim, temporal_dim),
                nn.Sigmoid()
            )

            self.classifier = nn.Sequential(
                nn.Linear(temporal_dim, 64),
                nn.ReLU(),
                nn.Linear(64, num_classes)
            )

        def forward(self, x, alarm_levels):
            batch_size = x.size(0)
            temporal_feat = self.temporal_encoder(x)
            channel_feat = self.channel_encoder(x.transpose(1, 2))

            temporal_feat = temporal_feat.permute(1, 0, 2)
            enhanced_feat = self.feature_aware(temporal_feat, alarm_levels)
            enhanced_feat = enhanced_feat.permute(1, 0, 2).mean(dim=1)
            channel_feat = channel_feat.mean(dim=1)

            fused = torch.cat([enhanced_feat, channel_feat], dim=1)
            gate_weight = self.gate(fused)
            fused_feat = gate_weight * enhanced_feat + (1 - gate_weight) *
channel_feat
            return self.classifier(fused_feat)


    def preprocess(data_path):
        try:
            df = pd.read_csv(data_path)
        except FileNotFoundError:
            raise FileNotFoundError(f"Data file {data_path} not found.")

        required_cols = ['数据采集时间', '电池单体电压最高值', '电池单体电压最低
值', 'SOC', '累计里程',
                         '绝缘电阻', '电机控制器输入电压', '电机控制器直流母
线电流', '充电状态',
                         '最高温度值', '最低温度值', '总电压', '总电流', '最高报
警等级']
```

```
        missing_cols = [col for col in required_cols if col not in df.columns]
        if missing_cols:
            raise ValueError(f"Missing required columns: {missing_cols}")

        df['时间差'] = pd.to_datetime(df['数据采集时间']).diff().dt.total_seconds() /
3600
        df['电压差异'] = df['电池单体电压最高值'] - df['电池单体电压最低值']
        df['SOC 变化率'] = df['SOC'].diff() / (df['时间差'] + 1e-5)
        df['温度变化率'] = df['最高温度值'].diff() / (df['时间差'] + 1e-5)
        df['电池健康指数'] = df['SOC'] / (df['累计里程'] + 1e-5) * df['绝缘电阻']
        df['电机效率波动'] = (df['电机控制器输入电压'] * df['电机控制器直流母线
电流']).rolling(10).std() / \
                                    ((df['电机控制器输入电压'] * df['电机控制器直流
母线电流']).rolling(10).mean() + 1e-5)
        df['充电温度一致性'] = df['最高温度值'].sub(df['最低温度值']).where(df['充
电状态'] == 1).rolling(10).mean()
        df['SOC-温度耦合系数'] = df['SOC'].rolling(10).corr(df['最高温度值'])
        df['急变功率指数'] = (df['总电压'] * df['总电流
']).diff().abs().rolling(10).mean()

        feature_cols = ['车辆状态', '充电状态', '车速', '累计里程', '总电压', '总电流',
'SOC',
                        'DC-DC 状态', '绝缘电阻', '驱动电机控制器温度', '驱动
电机转速',
                        '驱动电机转矩', '驱动电机温度', '电机控制器输入电压',
                        '电机控制器直流母线电流', '电池单体电压最高值', '电池
单体电压最低值',
                        '最高温度值', '最低温度值', '时间差', '电压差异', 'SOC 变
化率', '温度变化率',
                        '电池健康指数', '电机效率波动', '充电温度一致性',
'SOC-温度耦合系数', '急变功率指数']

        features = df[feature_cols].fillna(0).values
        labels = df['最高报警等级'].values
        scaler = StandardScaler()
        features = scaler.fit_transform(features)
        seq_data = np.concatenate([features, labels.reshape(-1, 1)], axis=1)
        return seq_data


    alarm_feature_map = {
        1: [17, 21, 10, 6, 22],
        2: [6, 13, 12, 5, 23],
        3: [10, 6, 17, 18, 20]
```

```
    }
    def predict_and_update(model, scaler, path, config, device, rank):
        df = pd.read_csv(path, encoding='gbk')
        seq_data, _ = preprocess(path)    # Preprocess without labels for prediction
        pred_dataset = EVDataset(seq_data, config['window_size'])
        pred_loader = DataLoader(pred_dataset, batch_size=config['batch_size']*2,
num_workers=config['num_workers'], pin_memory=True)

        model.eval()
        predictions = []
        with torch.no_grad():
            for batch_x, _, batch_alarm in pred_loader:
                batch_x = batch_x.to(device, non_blocking=True)
                batch_alarm = batch_alarm.to(device, non_blocking=True)
                with torch.cuda.amp.autocast():
                    outputs = model(batch_x, batch_alarm)
                    _, predicted = torch.max(outputs.data, 1)
                    predictions.extend(predicted.cpu().numpy())
    df.loc[config['window_size'] - 1:, '最高报警等级'] = predictions
        if rank == 0:    # Only rank 0 saves the file to avoid race conditions
            df.to_csv(path, index=False, encoding='gbk')
            print(f"预测结果已插入并保存至 {path}")
        return df
    def main():
        # 修改 1：使用 NCCL 后端
        dist.init_process_group(backend='nccl', init_method='env://')
        rank = dist.get_rank()
        local_rank = int(os.environ.get('LOCAL_RANK', 0))
        world_size = dist.get_world_size()

        config = {
            'batch_size': 256,
            'lr': 3e-3 * world_size,
            'epochs': 30,
            'window_size': 24,
            'input_dim': 28,
            'num_classes': 4,
            'num_workers': 12,
            'warmup_epochs': 2
        }

        # 修改 2：使用 CUDA 设备
        device = torch.device(f'cuda:{local_rank}')
        torch.cuda.set_device(device)
```

```
# 分布式数据加载（保持不变）
if rank == 0:
    full_data = preprocess("merged_cleaned_data_with_min_units.csv")
    data_tensor = torch.tensor(full_data, dtype=torch.float32)
    data_shape = torch.tensor(data_tensor.shape, dtype=torch.long)
else:
    data_shape = torch.empty((2,), dtype=torch.long)

dist.broadcast(data_shape, src=0)
if rank != 0:
    data_tensor = torch.empty(tuple(data_shape.tolist()), dtype=torch.float32)

dist.broadcast(data_tensor, src=0)
full_data = data_tensor.numpy()

train_size = int(0.8 * len(full_data))
train_data, val_data = full_data[:train_size], full_data[train_size:]

train_sampler = DistributedSampler(
    EVDataset(train_data, config['window_size']),
    num_replicas=world_size,
    rank=rank,
    shuffle=True
)

train_loader = DataLoader(
    EVDataset(train_data, config['window_size']),
    batch_size=config['batch_size'],
    sampler=train_sampler,
    num_workers=config['num_workers'],
    pin_memory=True,
    drop_last=True,
    persistent_workers=True
)

model = EnhancedDualTower(
    input_dim=config['input_dim'],
    window_size=config['window_size'],
    num_classes=config['num_classes'],
    alarm_feature_map=alarm_feature_map
).to(device)

model = DDP(model, device_ids=[local_rank], broadcast_buffers=False)
```

```
        optimizer = optim.AdamW([
            {'params': model.module.temporal_encoder.parameters()},
            {'params': model.module.channel_encoder.parameters(), 'lr': config['lr'] *
0.5},
            {'params': model.module.feature_aware.parameters(), 'lr': config['lr'] *
1.5}
        ], lr=config['lr'], weight_decay=0.01)

        # 修改 3：使用 CUDA 的调度器
        total_steps = config['epochs'] * len(train_loader)
        lr_scheduler = torch.optim.lr_scheduler.OneCycleLR(
            optimizer,
            max_lr=config['lr'],
            total_steps=total_steps,
            pct_start=0.3,
            anneal_strategy='cos'
        )

        criterion = nn.CrossEntropyLoss()
        # 修改 4：使用 CUDA 的 GradScaler
        scaler = torch.cuda.amp.GradScaler()

        best_val_acc = 0
        start_time = time.time()

        for epoch in range(config['epochs']):
            train_sampler.set_epoch(epoch)
            model.train()
            total_loss = 0
            epoch_start = time.time()

            for batch_x, batch_y, batch_alarm in train_loader:
                batch_x = batch_x.to(device, non_blocking=True)
                batch_y = batch_y.to(device, non_blocking=True)
                batch_alarm = batch_alarm.to(device, non_blocking=True)

                optimizer.zero_grad(set_to_none=True)

                # 修改 5：使用 CUDA 的自动混合精度
                with torch.cuda.amp.autocast():
                    outputs = model(batch_x, batch_alarm)
                    loss = criterion(outputs, batch_y)
```

```
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

            scaler.step(optimizer)
            scaler.update()
            lr_scheduler.step()

            total_loss += loss.item()

        total_loss_tensor = torch.tensor(total_loss, device=device)
        dist.all_reduce(total_loss_tensor, op=dist.ReduceOp.SUM)
        avg_loss = total_loss_tensor.item() / (len(train_loader) * world_size)

        if rank == 0 and (epoch + 1) % 2 == 0:
            val_loader = DataLoader(
                EVDataset(val_data, config['window_size']),
                batch_size=config['batch_size'] * 2,
                num_workers=config['num_workers']
            )

            model.eval()
            correct = 0
            total = 0
            with torch.no_grad():
                for batch_x, batch_y, batch_alarm in val_loader:
                    batch_x = batch_x.to(device)
                    batch_y = batch_y.to(device)
                    batch_alarm = batch_alarm.to(device)

                    with torch.cuda.amp.autocast():
                        outputs = model(batch_x, batch_alarm)
                    _, predicted = torch.max(outputs.data, 1)
                    total += batch_y.size(0)
                    correct += (predicted == batch_y).sum().item()

            val_acc = 100 * correct / total
            print(f"Epoch {epoch + 1}/{config['epochs']} | "
                  f"Loss: {avg_loss:.4f} | Val Acc: {val_acc:.2f}% | "
                  f"Time: {time.time() - epoch_start:.1f}s")

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                torch.save(model.module.state_dict(), 'best_model.pth')
```

```
        if rank == 0:
            total_time = time.time() - start_time
            print(f"Training completed in {total_time / 3600:.2f}h, best val acc:
{best_val_acc:.2f}%")
            if total_time > 7200:
                print("Warning: Training exceeded 2 hour limit!")

        dist.destroy_process_group()
    # Predict and update month_10.csv
        month_10_path = fr"{base_path}\month_10.csv"
        predict_and_update(model, scaler, month_10_path, config, device, rank)

        dist.destroy_process_group()

if __name__ == "__main__":
main()
```